

Go (Golang)

SDE2 Interview Guide

100 Questions & Answers with Code Examples

Level: Mid (2–4 yrs)	Stack: Go / Golang	Roles: SDE2 / Backend
----------------------	--------------------	-----------------------

#	Topic	Questions
1	Goroutines & Concurrency	Q1–Q15
2	Channels & Select	Q16–Q28
3	Runtime & Memory	Q29–Q40
4	Interfaces & Types	Q41–Q52
5	Error Handling	Q53–Q60
6	Context	Q61–Q67
7	Generics	Q68–Q74
8	Testing & Benchmarking	Q75–Q82
9	Go Patterns	Q83–Q92
10	Miscellaneous & Advanced	Q93–Q100

> **Focus:** OSI Model, TCP/IP, HTTP, DNS, TLS, Load Balancing, WebSockets, gRPC, Service Mesh, Go Networking | **Level:** SDE2

Table of Contents

[TCP/IP Deep Dive](#2-tcpip-deep-dive) — Q16–Q30
[HTTP/1.1, HTTP/2, HTTP/3](#3-http11-http2-http3) — Q31–Q45
[DNS & TLS/SSL](#4-dns-tlssl) — Q46–Q60
[Load Balancing & Proxies](#5-load-balancing-proxies) — Q61–Q75
[WebSockets, gRPC & Modern Protocols](#6-websockets-grpc-modern-protocols) — Q76–Q88
[Go Networking Patterns](#7-go-networking-patterns) — Q89–Q100

OSI Model & Fundamentals

Q1. What is the OSI model?

Difficulty: Easy

Layer 7: Application – HTTP, gRPC, DNS, SMTP, FTP
Layer 6: Presentation – TLS/SSL, encoding, compression
Layer 5: Session – session management (rarely distinct today)
Layer 4: Transport – TCP, UDP (ports, reliability)
Layer 3: Network – IP, ICMP, routing
Layer 2: Data Link – Ethernet, MAC addresses, switches
Layer 1: Physical – cables, NICs, signals

In practice: TCP/IP model collapses to 4 layers:

Application (L7+L6+L5)
Transport (L4: TCP/UDP)
Internet (L3: IP)
Link (L2+L1: Ethernet)

Data flow: App → TCP → IP → Ethernet → wire → reverse at destination

Each layer adds/removes header as data travels down/up the stack

Interview tip: "Most interview questions are about L4 (TCP/UDP) and L7 (HTTP). L3 (IP routing) matters for cloud infrastructure questions."

Q2. What is the difference between TCP and UDP?

Difficulty: Easy

TCP (Transmission Control Protocol):

Connection-oriented (3-way handshake before data)
Reliable: ACK, retransmit on loss
Ordered: sequence numbers ensure order
Flow control: receiver advertises window size
Congestion control: AIMD, slow start, etc.
Overhead: ~20 byte header + connection state
Use: HTTP, HTTPS, databases, file transfer, email

UDP (User Datagram Protocol):

Connectionless: no handshake
Unreliable: fire-and-forget, no ACK
Unordered: packets may arrive out of order
No flow/congestion control
Low overhead: 8 byte header
Use: DNS, video streaming, gaming, VoIP, QUIC (HTTP/3)

TCP

UDP

Reliability	Yes	No
Ordering	Yes	No
Speed	Slower	Faster
Use	Accuracy matters	Speed matters

QUIC (HTTP/3): UDP-based but adds reliability + security at app level
 Eliminates TCP head-of-line blocking
 0-RTT / 1-RTT connection establishment

Q3. What is the TCP three-way handshake?

Difficulty:
Medium

Establishes TCP connection before any data is sent:

```
Client → Server: SYN (seq=x)           "I want to connect"
Server → Client: SYN-ACK (seq=y, ack=x+1) "OK, and I want to connect too"
Client → Server: ACK (ack=y+1)         "Acknowledged"
```

Now: both sides have synchronized sequence numbers → data flow begins

Time cost: 1 RTT (round-trip time) before first byte of data

HTTPS adds TLS handshake on top:

```
TCP 3-way: 1 RTT
TLS 1.2: 2 more RTTs = 3 RTTs total before first byte
TLS 1.3: 1 more RTT = 2 RTTs total
TLS 1.3 0-RTT: 1 RTT total (for resumed sessions)
HTTP/3 (QUIC): 0-1 RTT (combines TCP+TLS handshake)
```

Connection teardown (4-way):

```
FIN → ACK → FIN → ACK
Half-close: one side can stop sending while other continues
```

TIME_WAIT state: 2×MSL (Max Segment Lifetime) after close
 Prevents old packets from being mistaken for new connection
 MSL = 30-120 seconds → lots of TIME_WAIT on busy servers
 Fix: SO_REUSEADDR or reduce TIME_WAIT with net.inet.tcp.tw_reuse

Q4. What is a socket and how does TCP use it?

Difficulty: Easy

Socket: OS abstraction for network communication endpoint
 Identified by: (protocol, local_IP:port, remote_IP:port)

Types:

```
SOCK_STREAM: TCP (reliable, ordered byte stream)
SOCK_DGRAM:  UDP (unreliable datagrams)
```

Server lifecycle:

```
socket() → bind(addr:port) → listen() → accept() → read/write → close()
```

Client lifecycle:

```
socket() → connect(server_addr:port) → read/write → close()
```

Socket options:

```
SO_REUSEADDR: allow reuse of port in TIME_WAIT (for quick server restart)
SO_REUSEPORT: allow multiple sockets on same port (load balancing)
TCP_NODELAY:  disable Nagle's algorithm (send small packets immediately)
```

SO_KEEPALIVE: detect dead connections
SO_RCVBUF/SO_SNDBUF: receive/send buffer sizes

Go:

```
net.Dial("tcp", "host:80") → *net.TCPConn (implements net.Conn)
net.Listen("tcp", ":8080") → net.Listener
listener.Accept() → net.Conn
```

Q5. What is a port and what are well-known ports?

Difficulty: Easy

Port: 16-bit number (0-65535) identifying a specific process/service

Categories:

0-1023: Well-known ports (require root/admin)
1024-49151: Registered ports (applications)
49152-65535: Dynamic/ephemeral ports (client-side connections)

Well-known ports:

20/21: FTP (data/control)
22: SSH
25: SMTP
53: DNS
80: HTTP
443: HTTPS
3306: MySQL
5432: PostgreSQL
6379: Redis
9092: Kafka
5672: RabbitMQ
2181: ZooKeeper
8080: Common alternative HTTP

Ephemeral ports:

Client uses random port (e.g., 52341) when connecting to server
OS assigns from ephemeral range
Maximum simultaneous connections from one IP: ~16K
(limited by 65535 - 49152 ephemeral ports)
But: limited by (src_ip, src_port, dst_ip, dst_port) uniqueness

Q6. What is NAT (Network Address Translation)?

Difficulty:
Medium

NAT: maps private IP:port → public IP:port for outbound connections
Enables many devices to share one public IP
Home router: 192.168.x.x → your ISP-assigned IP

Types:

SNAT (Source NAT): replace source IP:port (most common, outbound)
DNAT (Destination NAT): replace destination IP:port (port forwarding, inbound)

NAT translation table:

Private 192.168.1.100:52341 → Public 1.2.3.4:60000
Reply: 1.2.3.4:60000 → reverse translate → 192.168.1.100:52341

Problems with NAT:

Breaks end-to-end connectivity (server can't initiate to client)
WebSockets / long-lived connections: NAT timeout can kill them

UDP hole punching needed for P2P

Cloud load balancers use DNAT:

Client → LB_IP:443 → DNAT → backend_IP:8080

Backend sees LB IP as source (unless X-Forwarded-For added)

Q7. What is IP addressing (IPv4 vs IPv6)?

Difficulty: Easy

IPv4: 32-bit address (4.3 billion total)

Format: 192.168.1.1 (dotted decimal)

Exhausted: IANA allocated all /8 blocks in 2011

Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

IPv6: 128-bit address (340 undecillion total)

Format: 2001:0db8:85a3::8a2e:0370:7334 (colon-hex)

:: = consecutive zeros omitted

Loopback: ::1 (vs IPv4 127.0.0.1)

Link-local: fe80::/10

CIDR notation: 192.168.1.0/24

/24 = 24 bits mask = 256 addresses (192.168.1.0 - 192.168.1.255)

/16 = 65536 addresses

/8 = 16.7M addresses

Subnetting:

VPC CIDR: 10.0.0.0/16 → up to 65536 IPs

Subnet: 10.0.1.0/24 → 256 IPs

Reserved:

0.0.0.0/8: "this network"

127.0.0.0/8: loopback

169.254.0.0/16: link-local (APIPA, no DHCP)

224.0.0.0/4: multicast

255.255.255.255: broadcast

Q8. What is the difference between a router, switch, and hub?

Difficulty: Easy

Hub (L1): broadcasts all traffic to all ports

No intelligence, just repeats signal

Creates collision domain → everyone shares bandwidth

Obsolete

Switch (L2): forwards frames based on MAC address table

Learns: "MAC address X is on port 3" → unicast only to that port

Creates separate collision domains per port

Modern networks: all switches, no hubs

Router (L3): routes packets between networks based on IP address

Connects different subnets / networks

Maintains routing table (next-hop for each destination)

NAT, firewall, DHCP often colocated

Cloud equivalents:

Switch → VPC (Layer 2 isolation within VPC)

Router → VPC Router / Internet Gateway / NAT Gateway

Firewall → Security Groups, NACLs

Load Balancer → ALB (L7), NLB (L4)

L4 Load Balancer: routes by IP/TCP (like router)

L7 Load Balancer: routes by HTTP headers/URL (like application proxy)

Q9. What is BGP (Border Gateway Protocol)?

Difficulty:
Medium

BGP: routing protocol that glues the internet together

"Protocol of the internet" – routes between Autonomous Systems (ASes)

Each ISP, cloud provider = one AS (assigned AS number)

eBGP: between different ASes (internet routing)

iBGP: within same AS (internal routing)

Anycast: multiple servers share same IP address

BGP advertises same IP from multiple locations

Router sends traffic to "nearest" (fewest BGP hops)

Cloudflare 1.1.1.1 DNS: anycast → 200+ locations

How CDN works (anycast):

cdn.example.com → same IP everywhere

DNS resolves to anycast IP

BGP routes your request to nearest PoP

→ Low latency globally

BGP hijacking: AS advertises incorrect routes (accidental or malicious)

2008 Pakistan Telecom accidentally took YouTube offline

Fix: RPKI (Resource Public Key Infrastructure) for route validation

Q10. What is ICMP and how is it used?

Difficulty: Easy

ICMP (Internet Control Message Protocol):

Used by network devices for diagnostics and error messages

Operates at L3 (IP level, not TCP/UDP)

Common uses:

ping: ICMP Echo Request / Echo Reply → test connectivity + RTT

traceroute: sends packets with increasing TTL

TTL=1 → first hop returns ICMP Time Exceeded

TTL=2 → second hop returns ICMP Time Exceeded

... reveals each hop in path

ICMP message types:

0: Echo Reply (ping response)

8: Echo Request (ping)

3: Destination Unreachable (port closed, host down)

11: Time Exceeded (TTL=0, used by traceroute)

ICMP in firewalls:

Many firewalls block ICMP → ping fails but host is up

Traceroute may show * * * for firewall hops

Path MTU Discovery uses ICMP:

Sends large packet with DF (Don't Fragment) bit

Intermediate router sends "ICMP Fragmentation Needed" with max MTU

Sender reduces packet size to fit

Q11. ### Q11-Q15: Additional Networking Fundamentals**Difficulty:**
Medium

Q	Topic
Q11	What is MTU and how does fragmentation work?
Q12	What are VLANs and network segmentation?
Q13	What is a VPN and how does it work?
Q14	What is network bandwidth vs latency vs throughput?
Q15	What is network congestion and how is it detected?

TCP/IP Deep Dive

Q12. What is TCP flow control?**Difficulty: Hard**

Flow control: prevents fast sender from overwhelming slow receiver
Receiver advertises window size (how much it can accept)
Sender only sends up to window size bytes without ACK

Sliding window:

Sender maintains: send_base, next_seq_num, send_window
Can send: [send_base, send_base + window_size) without ACK
On ACK: slide window forward

Receive window (rwnd):

Advertised by receiver in TCP header (16 bits → max 65535 bytes)
Window scaling option (TCP option): up to 1GB window
If rwnd = 0: sender stops (zero window probe to detect recovery)

Congestion window (cwnd): sender-side congestion control

Limits sending rate based on network capacity
Separate from flow control (minimum of both applies)

Throughput = window_size / RTT

1MB window, 100ms RTT → max 80 Mbps throughput
10MB window, 10ms RTT → max 8 Gbps throughput

Buffer bloat:

Huge buffers in network equipment → high latency when congested
Packets sit in queue for seconds → bad for real-time apps
Fix: AQM (Active Queue Management), CoDel, FQ-CoDel

Q13. What is TCP congestion control?**Difficulty: Hard**

Congestion control: prevent sender from overwhelming network
4 algorithms in standard TCP:

1. Slow Start:

cwnd starts at 1 MSS (max segment size $\approx$ 1460 bytes)
 Double cwnd each RTT (exponential growth)
 Until: cwnd > ssthresh (slow start threshold)

2. Congestion Avoidance:

cwnd > ssthresh $\rightarrow$ grow by 1 MSS per RTT (linear)
 "Additive Increase"

3. Fast Retransmit:

3 duplicate ACKs $\rightarrow$ packet lost $\rightarrow$ retransmit immediately (don't wait timeout)

4. Fast Recovery (TCP Reno/CUBIC):

On 3 dup ACKs: ssthresh = cwnd/2, cwnd = ssthresh + 3
 Continue with congestion avoidance (don't go back to slow start)

On timeout: ssthresh = cwnd/2, cwnd = 1 MSS (full slow start)

Modern: CUBIC (default Linux), BBR (Google)

CUBIC: window growth based on time since last congestion

BBR: estimates bottleneck bandwidth and RTT (not loss-based)

BBR: significantly better for high-bandwidth, long-latency (intercontinental)

Q14. What is TCP keepalive?

Difficulty:
Medium

TCP keepalive: detect dead connections without data transfer

Problem: NAT gateway, firewalls close idle TCP connections silently

Result: application thinks connection is alive, packets go to dead NAT entry

Without keepalive: NAT timeout (30min-24hrs) kills connection silently

Application sends data $\rightarrow$ connection error $\rightarrow$ reconnect needed

With TCP keepalive:

SO_KEEPALIVE socket option

OS sends keepalive probes after idle time

Parameters: tcp_keepalive_time=7200, tcp_keepalive_intvl=75, tcp_keepalive_probes=9

Default (7200s idle, 75s probe): too slow for most apps

Application-level keepalive (better):

WebSocket: ping/pong frames

HTTP/2: PING frames

gRPC: HTTP/2 PING

Custom protocols: heartbeat messages

// Go HTTP client keepalive:

```
transport := &http.Transport{
    IdleConnTimeout:      90 * time.Second,
    DisableKeepAlives:    false,
    MaxIdleConns:         100,
    MaxIdleConnsPerHost: 10,
    TLSHandshakeTimeout: 10 * time.Second,
}
```

Q15. What is TCP TIME_WAIT and why does it exist?

Difficulty: Hard

TIME_WAIT: state after active close ($2 \times$ MSL duration)

MSL (Maximum Segment Lifetime) $\approx$ 30-60 seconds
 TIME_WAIT duration: 60-120 seconds typical

Why TIME_WAIT exists:

1. Ensure final ACK reaches other side
 If FIN arrives after connection closed, need to send ACK
2. Prevent stale packets from old connection
 Old delayed packet arrives on reused port $\rightarrow$ might be mistaken for new connection
 TIME_WAIT: old packets expire before port is reused

Problem: millions of short-lived connections $\rightarrow$ TIME_WAIT sockets accumulate
 ss -s | grep TIME_WAIT $\rightarrow$ thousands $\rightarrow$ ephemeral port exhaustion

Solutions:

SO_REUSEADDR: reuse address in TIME_WAIT (usually enabled by default)
 net.ipv4.tcp_tw_reuse = 1: reuse TIME_WAIT sockets for new connections
 net.ipv4.tcp_fin_timeout = 15: reduce to 15s (from 60s default)
 Keep connections alive (connection pool): avoid rapid connect/close

// Go server: SO_REUSEADDR is automatic via net.Listen
 // Go client: use persistent connections (http.Transport with MaxIdleConns)

Q16. What is Nagle's algorithm?

Difficulty:
Medium

Nagle's algorithm: buffer small TCP segments to reduce packet count
 If unACKed data in flight: buffer new data until ACK arrives
 Result: small writes are coalesced into larger packets

Good for: bulk transfer (fewer packets, better throughput)
 Bad for: real-time/interactive apps (adds latency)

TCP_NODELAY: disables Nagle's algorithm (send immediately)
 Essential for: low-latency applications, games, real-time data
 HTTP/1.1: often set TCP_NODELAY (each request is independent)
 gRPC: uses TCP_NODELAY (over HTTP/2)

Interaction with Delayed ACK:

Receiver delays ACK (up to 40ms) hoping to piggyback on response
 + Nagle's algorithm waiting for ACK
 = "ACK delay" problem: 40ms latency on small bidirectional messages
 Fix: TCP_NODELAY on both sides

```
// Go: disable Nagle's algorithm
conn, _ := net.Dial("tcp", "host:port")
tcpConn := conn.(*net.TCPConn)
tcpConn.SetNoDelay(true)
```

// HTTP server: automatically uses TCP_NODELAY (net/http default)

Q17. What is connection pooling at the TCP level?

Difficulty:
Medium

Problem: TCP connection setup = 1-3 RTT (handshake + TLS)
 At 100ms RTT: 200-300ms per new connection
 At 1000 QPS: 1000 $\times$ 300ms overhead = impractical

Connection pool: maintain N pre-established TCP connections, reuse

Benefits:

Reuse: no handshake overhead per request
 Concurrency: multiple in-flight requests on same connection (HTTP/2 multiplexing)
 Connection limit: controlled max connections to downstream services

Go http.Transport (built-in pool):

```
transport := &http.Transport{
    MaxIdleConns:    100,    // total pool size
    MaxIdleConnsPerHost: 10,    // per-host pool
    MaxConnsPerHost:  0,    // unlimited total per host (0=no limit)
    IdleConnTimeout:  90 * time.Second, // close idle after 90s
    DisableKeepAlives: false, // MUST be false for pooling
}
client := &http.Client{Transport: transport}
```

Database pool (pgxpool):

Maintains persistent TCP connections to PostgreSQL
 Requests borrow connection → return to pool after use

gRPC connection pooling:

gRPC uses HTTP/2 multiplexing: many RPC calls on one connection
 Multiple goroutines share one TCP connection

Q18. What is TCP SYN flood and how do you defend against it?**Difficulty: Hard**

SYN flood: DDoS attack exhausting server's connection table

Normal SYN: Client SYN → Server allocates half-open connection state → Server SYN-ACK

SYN flood: Attacker sends 1M SYNs → Server allocates 1M half-open connections
 → Server memory exhausted → legitimate connections refused

Half-open: Server in SYN_RCVD state waiting for final ACK

Default limit: ~8192 half-open connections (depends on OS)

Attack: spoof source IPs → Server never gets ACK → SYN timeout (75s)

Defense:**1. SYN cookies (most important):**

Server doesn't allocate state for SYN
 Encodes connection info in sequence number (cryptographic)
 Full state allocated only when ACK arrives
 Linux default: `net.ipv4.tcp_syncookies = 1`

2. Firewall rate limiting:

`iptables`: limit SYN packets per second per source IP

3. Increase backlog queue:

`net.ipv4.tcp_max_syn_backlog = 8192 → 65536`

4. Reduce SYN-ACK retries:

`net.ipv4.tcp_synack_retries = 2` (default 5)

5. DDoS protection services: Cloudflare, AWS Shield, Fastly**Q19. What is the difference between connection-oriented and connectionless?****Difficulty: Easy**

Connection-oriented (TCP):

- Establish connection before data (3-way handshake)
- State maintained at both endpoints
- Guaranteed delivery, ordering, flow control
- Use: HTTP, database connections, file transfer

Connectionless (UDP):

- No connection establishment
- Each datagram independent (no state)
- Best-effort delivery, no ordering
- Use: DNS queries, video streaming, gaming

Connectionless advantages:

- Low overhead: no handshake RTT, no state memory
- Simple: fire and forget
- Broadcast/multicast: easy (TCP is unicast)

When latency > reliability:

- Video streaming: dropped frame → better than waiting for retransmit
- Gaming: stale position update → useless, just use latest
- DNS: single request/response, retry is simple

QUIC: UDP-based but adds reliability selectively

- Per-stream reliability: losing one stream doesn't block others (no HoL blocking)
- Best of both worlds: connection semantics + connectionless flexibility

Q20. What is half-duplex vs full-duplex?**Difficulty: Easy**

Half-duplex: transmit OR receive at a time (not simultaneously)

Example: walkie-talkies, old Ethernet hubs

Full-duplex: transmit AND receive simultaneously

Example: TCP connections, telephone calls, modern Ethernet

TCP is full-duplex:

- Client → Server (data stream)
- Server → Client (data stream)
- Both simultaneously on same TCP connection

Application protocols:

- HTTP/1.1: request → response → request (half-duplex at app level)
- HTTP/2: full multiplexed (request and response simultaneously)
- WebSocket: full-duplex bidirectional messaging
- gRPC streaming: full-duplex bidirectional RPCs

Hardware:

- 10BASE-T: half-duplex (one hub, shared medium)
- 100BASE-TX with switch: full-duplex (dedicated segment per port)
- Modern networks: almost always full-duplex

Q21. What are TCP socket options that matter for backend systems?**Difficulty: Hard****SO_REUSEADDR:**

- Allow binding to address in TIME_WAIT
- Essential for server restart without waiting
- Go: automatic with net.Listen

SO_REUSEPORT:

Allow multiple sockets on same port
 Kernel load-balances incoming connections
 Use: multi-process servers (NGINX workers)
 Go: requires `syscall.SetsockoptInt`

TCP_NODELAY:

Disable Nagle's algorithm
 Essential for low-latency protocols
 Go: `tcpConn.SetNoDelay(true)`

SO_KEEPALIVE:

Detect dead connections via kernel-level probes
 Go: `tcpConn.SetKeepAlive(true); tcpConn.SetKeepAlivePeriod(30*time.Second)`

SO_LINGER:

Control what happens to unsent data on `close()`
`linger=0`: RST immediately (no graceful close)
`linger=N`: wait up to N seconds for data to send

SO_RCVBUF / SO_SNDBUF:

Socket send/receive buffer sizes
 Default ~128KB; increase to 4MB+ for high-bandwidth long-latency links

TCP_FASTOPEN:

Send data in SYN packet (0-RTT for repeated connections)
 Server: `setsockopt TCP_FASTOPEN`
 Saves 1 RTT for reconnections with same server

Q22. ### Q26-Q30: More TCP/IP Topics**Difficulty:**
Medium

Q	Topic
Q26	TCP retransmission and RTO (Retransmit Timeout) calculation
Q27	IP fragmentation and PMTU discovery
Q28	TCP window scaling and BDP (Bandwidth-Delay Product)
Q29	TCP head-of-line blocking explained
Q30	Netstat, ss, tcpdump for network debugging

HTTP/1.1, HTTP/2, HTTP/3

Q23. What is HTTP/1.1 and what are its limitations?**Difficulty:** Easy

HTTP/1.1 (1997): text-based request/response protocol
 Each request = one TCP connection (or persistent with Keep-Alive)
 Keep-Alive: reuse connection for multiple requests (sequential)

Request format:

`GET /users/42 HTTP/1.1`

```
Host: api.example.com
Authorization: Bearer token...
Accept: application/json
\r\n
```

Response format:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 123
\r\n
{"id": 42, "name": "Alice"}
```

HTTP/1.1 limitations:

- Head-of-line blocking: requests are sequential per connection
 - Request 2 blocked until Response 1 completes
- Text headers: verbose, no compression
- No server push: server must wait for request

Workarounds (pre-HTTP/2 era):

- Multiple connections (browsers: 6 per domain)
- Domain sharding: spread assets across subdomains
- Concatenation: bundle JS/CSS into one file
- Spriting: combine images into one sprite
 - All unnecessary with HTTP/2

Q24. What is HTTP/2 and how does it improve HTTP/1.1?

Difficulty:
Medium

HTTP/2 (2015): binary framing layer, same semantics as HTTP/1.1

Key improvements:

- Multiplexing: multiple streams on one TCP connection
 - Stream: logical channel identified by stream ID (odd=client, even=server)
 - Streams interleaved: no head-of-line blocking at HTTP level
 - 100 requests → one TCP connection, parallel responses
- Header compression (HPACK):
 - Static table: common headers (e.g., :method: GET)
 - Dynamic table: previous request headers reused
 - Typical reduction: 85-95% header size
- Binary framing: structured binary (vs text)
 - Efficient parsing, no ambiguity
- Server Push: server proactively sends resources
 - "You'll need index.css, I'll send it without you asking"
 - PUSH_PROMISE frame
- Stream prioritization: weight + dependency tree
 - Critical resources get more bandwidth

Remaining issue: TCP head-of-line blocking

- HTTP/2 multiplexes over ONE TCP connection
- If TCP packet lost: ALL HTTP/2 streams stall until retransmit
 - HTTP/3 (QUIC) solves this with UDP

Q25. What is HTTP/3 (QUIC)?**Difficulty: Hard**

HTTP/3 (2022): HTTP/2 semantics over QUIC (not TCP)

QUIC (Quick UDP Internet Connections):

Built on UDP

Implements reliability, ordering, congestion control at application level

Per-stream reliability: losing stream 3 doesn't block stream 4

Key improvements over HTTP/2:

1. No TCP head-of-line blocking:

Each stream independent → packet loss only affects that stream

2. Faster connection establishment:

TCP+TLS 1.3: 1 RTT (vs 2 RTT for TCP+TLS 1.2)

QUIC+TLS 1.3: 1 RTT (combined transport+crypto handshake)

QUIC 0-RTT: 0 RTT for known servers (session resumption)

3. Connection migration:

Connection ID (not IP:port)

Switch WiFi → cellular → connection survives (no reconnect)

4. Built-in encryption (always TLS 1.3):

QUIC mandates encryption; no unencrypted mode

5. Reduced middlebox interference:

NAT/firewalls can't inspect/tamper with QUIC stream layer

Adoption:

Google: ~50% of Google traffic is HTTP/3

Cloudflare, Fastly: support HTTP/3

Go 1.21: HTTP/3 via golang.org/x/net/quit (experimental)

Popular library: quic-go

Q26. What are HTTP methods and when do you use each?**Difficulty: Easy**

GET: Retrieve resource. Safe + idempotent. Cacheable.

POST: Create resource / submit data. Not idempotent. Not cacheable by default.

PUT: Replace resource entirely. Idempotent.

PATCH: Partial update. Not necessarily idempotent.

DELETE: Remove resource. Idempotent.

HEAD: Like GET but no body. Used to check existence/metadata.

OPTIONS: List allowed methods. Used by CORS preflight.

CONNECT: Establish tunnel (HTTPS through proxy).

TRACE: Echo request (debugging). Usually disabled.

Safe: read-only (no side effects) – GET, HEAD, OPTIONS

Idempotent: repeat calls = same state – GET, PUT, DELETE, HEAD, OPTIONS

REST convention:

GET /users/42 → get user

POST /users → create user

PUT /users/42 → replace user

PATCH /users/42 → update user fields

DELETE /users/42 → delete user

PATCH vs PUT:

PUT: send entire object (replace)
PATCH: send only changed fields (merge)
Use PATCH for partial updates to avoid overwriting concurrent changes

Q27. What are HTTP status codes?**Difficulty: Easy****1xx Informational:**

100 Continue: client should continue sending request body
101 Switching Protocols: upgrading to WebSocket

2xx Success:

200 OK: standard success
201 Created: resource created (POST response, include Location header)
204 No Content: success with no body (DELETE, PATCH with no response needed)
206 Partial Content: range request (streaming video)

3xx Redirection:

301 Moved Permanently: redirect forever (browser caches)
302 Found: temporary redirect (browser doesn't cache)
304 Not Modified: ETag/Last-Modified matched, use cache
307 Temporary Redirect: like 302 but MUST NOT change method
308 Permanent Redirect: like 301 but MUST NOT change method

4xx Client Error:

400 Bad Request: malformed request
401 Unauthorized: authentication required
403 Forbidden: authenticated but not authorized
404 Not Found: resource doesn't exist
405 Method Not Allowed: wrong HTTP method
409 Conflict: resource state conflict (duplicate)
422 Unprocessable Entity: validation failed
429 Too Many Requests: rate limit hit (include Retry-After)

5xx Server Error:

500 Internal Server Error: unexpected server error
502 Bad Gateway: upstream server failure (LB/proxy)
503 Service Unavailable: server overloaded or down
504 Gateway Timeout: upstream timed out

Q28. What are HTTP headers you should know?**Difficulty: Easy****Request headers:**

Authorization: Bearer token / Basic base64(user:pass)
Content-Type: application/json / multipart/form-data
Accept: application/json, text/html (what I accept)
Accept-Encoding: gzip, deflate, br (compression)
Cache-Control: no-cache / max-age=0
If-None-Match: "etag-value" (conditional GET)
If-Modified-Since: Thu, 01 Jan 2024 00:00:00 GMT
X-Request-ID: uuid (tracing)
X-Forwarded-For: client-ip (behind proxy)
User-Agent: Mozilla/5.0 ...
Host: api.example.com (required in HTTP/1.1)
Idempotency-Key: uuid (payment dedup)

Response headers:

```
Content-Type: application/json; charset=utf-8
Content-Length: 1234
Content-Encoding: gzip
Cache-Control: public, max-age=3600, s-maxage=86400
ETag: "abc123" (resource version for caching)
Last-Modified: Thu, 01 Jan 2024 00:00:00 GMT
Location: /users/42 (after POST 201)
Retry-After: 60 (after 429 or 503)
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 950
Strict-Transport-Security: max-age=31536000; includeSubDomains
Vary: Accept-Encoding (tells CDN to cache different versions)
```

Q29. What is content negotiation?

Difficulty:
Medium

Content negotiation: client and server agree on response format/language

Request: client sends preferences

```
Accept: application/json, text/html;q=0.9, */*;q=0.8
Accept-Language: en-US,en;q=0.9,fr;q=0.5
Accept-Encoding: gzip, deflate, br
```

q value: quality factor (0-1, default 1.0)
Higher q = more preferred

Server: picks best match, sends:

```
Content-Type: application/json
Content-Language: en
Content-Encoding: gzip
Vary: Accept, Accept-Language (vary header tells CDN)
```

Proactive vs reactive:

Proactive: server selects representation based on preferences
Reactive: server sends multiple choices, client selects (301 Multiple Choices)

API versioning via Accept:

```
Accept: application/vnd.myapi.v2+json
→ Server serves API v2 format
```

Response 406 Not Acceptable:

Server cannot serve any of client's preferred formats

Q30. What is HTTP caching?

Difficulty:
Medium

HTTP caching layers: browser, CDN, reverse proxy

Cache-Control directive:

```
max-age=3600: cache for 1 hour
s-maxage=86400: CDN caches for 1 day (overrides max-age for shared caches)
no-cache: revalidate before serving (can still cache)
no-store: never cache (sensitive data)
public: cacheable by any cache (CDN)
private: only browser cache (user-specific data)
must-revalidate: once expired, must revalidate before serving stale
stale-while-revalidate=60: serve stale up to 60s while refreshing async
```

Conditional requests (validation):

Server sends: ETag: "abc123" or Last-Modified: date
Client sends: If-None-Match: "abc123" or If-Modified-Since: date
Server: 304 Not Modified (no body) → client uses cached version
Server: 200 OK + new body → content changed

Cache busting:

Versioned URL: /static/app.v123.js → change version on deploy
Query param: /api/config?v=2024010 → change on deploy

Vary header:

Vary: Accept-Encoding → CDN caches separate versions for gzip vs non-gzip
Vary: Authorization → don't cache (each user different data)

Q31. What is CORS (Cross-Origin Resource Sharing)?**Difficulty:**
Medium

CORS: browser security mechanism preventing cross-origin requests by default

Same-origin: same scheme + host + port

http://example.com/api ← same as http://example.com/page
https://api.example.com ← different from https://example.com

Preflight request (OPTIONS):

For non-simple requests (non-GET/POST, custom headers, JSON body)
Browser: OPTIONS /api/users → server must respond with CORS headers
If server says OK → browser sends actual request

CORS headers:

Access-Control-Allow-Origin: https://app.example.com (or * for public)
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true (for cookies/auth)
Access-Control-Max-Age: 86400 (cache preflight for 24h)

CORS only in BROWSER:

Server-to-server: no CORS check (only browsers enforce it)
curl, Postman: no CORS restrictions

// Go CORS middleware:

```
w.Header().Set("Access-Control-Allow-Origin", origin)
w.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS")
w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
if r.Method == "OPTIONS" { w.WriteHeader(204); return }
```

Q32. What is HTTP Keep-Alive and connection reuse?**Difficulty:**
Medium

HTTP Keep-Alive: reuse TCP connection for multiple HTTP requests

HTTP/1.0: close after each request (default)
HTTP/1.1: keep-alive by default
HTTP/2: always multiplexed (one connection per server)

Headers:

Connection: keep-alive (HTTP/1.1 default)
Connection: close (close after response)
Keep-Alive: timeout=90, max=1000

Benefits:

Eliminate TCP + TLS handshake per request
 Pipelining (HTTP/1.1 extension): send multiple requests before responses
 HTTP/2 multiplexing: better, non-blocking

Go http.Client pool settings:

```
DefaultTransport.MaxIdleConns = 100
DefaultTransport.MaxIdleConnsPerHost = 2 (often too low!)
DefaultTransport.IdleConnTimeout = 90s
```

// Increase for high-concurrency services:

```
transport := &http.Transport{
    MaxIdleConns:      1000,
    MaxIdleConnsPerHost: 100,
    IdleConnTimeout:    90 * time.Second,
}
// ALWAYS reuse http.Client (don't create new one per request!)
var httpClient = &http.Client{Transport: transport}
```

Q33. ### Q41-Q45: More HTTP Topics**Difficulty:**
Medium

Q	Topic
Q41	HTTP/2 server push: use cases and limitations
Q42	HTTP compression: gzip vs brotli vs zstd
Q43	HTTP long polling vs SSE (Server-Sent Events) vs WebSocket
Q44	HTTP range requests for large file downloads
Q45	API versioning via URL vs header vs content negotiation

DNS & TLS/SSL

Q34. How does DNS resolution work?**Difficulty:**
Medium

DNS: translates domain names → IP addresses

Resolution chain:

Browser checks cache → OS cache → /etc/hosts → Recursive resolver →
 Root nameserver → TLD nameserver (.com) → Authoritative nameserver

Recursive resolver (your ISP or 8.8.8.8):

1. Check its own cache
2. Ask root nameserver: "where is .com?"
3. Root says: "ask 192.5.6.30 (com TLD)"
4. Ask com TLD: "where is example.com?"
5. TLD says: "ask 205.251.196.1 (example.com's NS)"
6. Ask authoritative NS: "what is api.example.com?"
7. Gets A record: 93.184.216.34 → return to client

DNS record types:

A: domain → IPv4 address
AAAA: domain → IPv6 address
CNAME: alias → another domain name
MX: domain → mail server + priority
NS: domain → nameserver name
TXT: text (SPF, DKIM, verification)
SRV: service discovery: host + port + priority
PTR: reverse DNS: IP → domain

TTL: how long to cache the answer

Low TTL (60s): fast failover, more DNS queries

High TTL (86400s): fewer queries, slow failover

Production: 300s (5 min) balances both

Q35. What is DNS load balancing and GeoDNS?

Difficulty:
Medium

Round-robin DNS: return multiple IPs, rotate order

nslookup api.example.com → 1.2.3.4, 5.6.7.8, 9.10.11.12

Client uses first IP → natural load distribution

Problem: no health checks (routes to dead servers)

GeoDNS: return different IPs based on client location

EU clients → EU servers (lower latency)

US clients → US servers

Route53 Latency Routing: returns lowest-latency endpoint

Route53 Geolocation: returns endpoint for user's country/region

Cloudflare Load Balancing: GeoDNS + health checks

DNS failover:

Health check monitors endpoints

Failed endpoint → remove from DNS

TTL determines how long until clients see change

Low TTL (30-60s) for faster failover

Anycast (BGP-based, not DNS):

Same IP advertised from multiple locations

BGP routing → nearest location handles request

Cloudflare, Fastly, Google DNS use anycast

Failover: instant (BGP reconverges in seconds)

Route53 routing policies:

Simple, Failover, Geolocation, Geoproximity,

Latency-based, Multivalue, Weighted, IP-based

Q36. What is DNS TTL and its impact?

Difficulty: Easy

TTL: Time To Live – how long DNS resolvers cache the answer

Low TTL (30-60 seconds):

Pros: fast failover (DNS change propagates in 30s)

Cons: more DNS queries (costs + latency)

Use: before planned maintenance, A/B testing

High TTL (3600-86400 seconds):

Pros: fewer DNS queries, cached longer

Cons: slow change propagation (up to TTL duration)
Use: stable infrastructure

TTL change strategy:

48h before cutover: lower TTL to 300s
Do cutover: DNS update propagates in 5 minutes
After cutover: raise TTL back to 3600s+

DNS caching layers:

Browser: caches up to TTL (Chrome: min 1s, max 5min regardless of TTL)
OS: respects TTL
Resolver: respects TTL

Negative TTL (NXDOMAIN):

SOA record's minimum field or NXDOMAIN TTL
How long to cache "this domain doesn't exist"
Default: 300-3600s
Impact: after creating new DNS record, takes TTL for clients to see it

Q37. What is TLS/SSL and how does the handshake work?

Difficulty: Hard

TLS (Transport Layer Security): encryption + authentication for TCP

TLS 1.3 Handshake (1 RTT):

Client → Server: ClientHello (TLS version, cipher suites, key share)
Server → Client: ServerHello + Certificate + CertificateVerify + Finished
(includes server's public key share)
Client → Server: Finished
Both: derive session keys → encrypted data flows

Key exchange: ECDHE (Ephemeral Elliptic Curve Diffie-Hellman)
Forward secrecy: even if private key compromised, past sessions safe

TLS 1.3 0-RTT (session resumption):

Client remembers PSK (Pre-Shared Key) from previous session
Client → Server: ClientHello + 0-RTT data (encrypted with PSK)
Server → Client: response + new session keys
Risk: replay attacks possible for 0-RTT data

TLS 1.2 Handshake (2 RTT):

Old, being phased out
No forward secrecy unless using ECDHE cipher suite

Certificates:

X.509 certificate: server's identity + public key, signed by CA
Chain: Server cert → Intermediate CA → Root CA
Browsers trust Root CAs (Mozilla, Google root store)

Certificate verification:

Hostname validation: cert matches requested hostname
OCSP / CRL: is cert revoked?
Certificate pinning: only trust specific cert/CA

Q38. What is mTLS (Mutual TLS)?

Difficulty: Hard

Standard TLS: client verifies server identity (server cert)

mTLS: both sides verify each other (client cert + server cert)

Use case:

- Service-to-service authentication in microservices
- Zero-trust networking: no implicit trust based on network location
- API security: strong client authentication
- Service mesh (Istio/Linkerd): auto-inject mTLS for all pod communication

mTLS flow:

- Both sides exchange certificates
- Both verify the other's certificate against trusted CA

Benefits:

- No passwords or API keys needed
- Certificate is the identity
- Automatic cert rotation with cert manager

Challenges:

- Certificate lifecycle management
- Rotation without downtime
- Debugging (harder than simple TLS)

Service mesh approach:

- Sidecar proxy (Envoy) handles mTLS automatically
- App doesn't need to implement TLS
- Cert manager (cert-manager.io) auto-rotates certs
- SPIFFE/SPIRE: standard for workload identity

```
// Go mTLS client:
cert, _ := tls.LoadX509KeyPair("client.crt", "client.key")
tlsConfig := &tls.Config{
    Certificates: []tls.Certificate{cert},
    RootCAs:      certPool,
}
transport := &http.Transport{TLSClientConfig: tlsConfig}
```

Q39. ### Q51-Q60: More DNS & TLS Topics

Difficulty:
Medium

Q	Topic
Q51	HTTPS certificate types: DV, OV, EV, wildcard
Q52	Certificate transparency (CT) logs
Q53	HSTS (HTTP Strict Transport Security)
Q54	SNI (Server Name Indication) for virtual hosting with TLS
Q55	Let's Encrypt and ACME protocol for auto-cert renewal
Q56	DNS over HTTPS (DoH) and DNS over TLS (DoT)
Q57	DNSSEC: signing DNS records to prevent spoofing
Q58	Certificate pinning: risks and alternatives
Q59	TLS session resumption: session IDs vs session tickets

Q60

OpenSSL vs Go crypto/tls: key differences

Load Balancing & Proxies

Q40. What is a load balancer and what are its algorithms?

Difficulty: Easy

Load balancer: distributes traffic across multiple backend servers

Layer 4 (Transport): routes by IP/TCP (fast, no content inspection)
AWS NLB, HAProxy TCP mode

Layer 7 (Application): routes by HTTP headers, URL, cookies (flexible)
AWS ALB, Nginx, HAProxy HTTP mode

Algorithms:

Round Robin: requests distributed evenly in sequence
Pro: simple. Con: ignores server load.

Weighted Round Robin: more requests to higher-weight servers
Use: heterogeneous servers (different capacity)

Least Connections: route to server with fewest active connections
Pro: handles variable request duration
Con: requires connection tracking

Least Response Time: route to fastest-responding server
Requires health check with latency measurement

IP Hash: $\text{hash}(\text{client_IP}) \% N \rightarrow$ sticky sessions
Same client $\rightarrow$ same server
Problem: mobile IPs change $\rightarrow$ breaks stickiness

Consistent Hash: minimal remapping when servers added/removed
Good for: cache servers (minimize cache misses on scaling)

Random: random server selection
Surprisingly competitive with Round Robin in practice

Health checks:

Active: LB polls /health every 10s $\rightarrow$ mark unhealthy if 3 failures
Passive: detect failures from real traffic (5xx, timeouts)

Q41. What is the difference between a reverse proxy and forward proxy?

Difficulty: Easy

Forward proxy: sits in front of CLIENTS

Client $\rightarrow$ Forward Proxy $\rightarrow$ Internet

Use: corporate firewall, VPN, content filtering, bypass geo-blocks

Client configured to use proxy

Server doesn't know client's real IP

Reverse proxy: sits in front of SERVERS

Internet $\rightarrow$ Reverse Proxy $\rightarrow$ Backend Servers

Use: load balancing, SSL termination, caching, security
Client doesn't know about backend servers
Server doesn't see client directly (X-Forwarded-For header)

Reverse proxy functions:

SSL termination: decrypt HTTPS once, forward HTTP internally
Load balancing: distribute requests
Caching: serve static content, cache responses
Compression: gzip responses
Rate limiting: per-IP request limiting
Security: hide backend topology, WAF

Examples:

Nginx: reverse proxy + load balancer + static serving
HAProxy: high-performance TCP/HTTP load balancer
Envoy: L7 proxy with rich observability (used in service mesh)
Traefik: cloud-native reverse proxy (Kubernetes-native)
AWS ALB/NLB: managed cloud load balancers
Cloudflare: global reverse proxy + CDN + DDoS protection

Q42. What is sticky sessions (session affinity)?

Difficulty:
Medium

Sticky sessions: route all requests from one client to same backend server
Needed when: server-side session state not shared across servers

Methods:

1. Cookie-based (most common):
LB sets cookie: SERVERID=server2
Subsequent requests: route based on cookie value
2. IP-based:
hash(client_IP) % N → always same server
Problem: mobile IPs change, NAT makes many clients look like one
3. Custom header:
App sets X-Server-ID header → LB routes based on it

Problems with sticky sessions:

Uneven load: popular "sticky" servers get more traffic
Failover: server dies → all sticky sessions lost anyway
Scaling: adding server doesn't immediately balance load

Better alternative: stateless services

Move session state to Redis (shared)
Any server can handle any request
No sticky sessions needed
True horizontal scaling

AWS ALB sticky sessions:

Duration-based: cookie with TTL
Application-based: app controls stickiness via its own cookie

Q43. What is SSL offloading/termination?

Difficulty:
Medium

SSL termination: decrypt HTTPS at the load balancer; forward HTTP internally

Architecture:

Client → HTTPS → [LB: decrypt TLS] → HTTP → Backend servers

Benefits:

Backend servers: simpler (no TLS management)
 Certificate management: one place (LB)
 CPU offload: TLS is CPU-intensive; LB has dedicated crypto hardware
 Performance: backend traffic on fast internal network (no TLS overhead)

Security consideration:

Internal network: HTTP (unencrypted)
 If internal network is untrusted: use SSL passthrough or re-encryption

SSL passthrough: LB forwards encrypted traffic directly to backend

Backend handles TLS itself
 LB cannot inspect content (no L7 routing by URL)
 Use: end-to-end encryption requirement

SSL re-encryption (bridging):

LB terminates TLS from client → re-encrypts to backend
 Full encryption everywhere
 LB can inspect and route (L7 capable)

AWS: ALB terminates TLS, forwards to ECS tasks via HTTP

NLB: TCP passthrough (no TLS termination)

Q44. What is health checking for load balancers?

Difficulty:
Medium

Health checks: LB probes backends to detect failures

Types:

TCP: connect to port (is server accepting connections?)
 HTTP: GET /health → expect 200 OK
 HTTPS: same as HTTP but with TLS
 gRPC: health check RPC (grpc.health.v1.Health/Check)
 Custom: run specific script/command

Configuration:

Check interval: every 10-30s
 Timeout: expect response within 5s
 Healthy threshold: 2 consecutive successes → mark healthy
 Unhealthy threshold: 3 consecutive failures → mark unhealthy

Health endpoint types:

Liveness: is the process alive? (binary: yes/no)
 Readiness: is the process ready to serve traffic?
 Checks: DB connection, Redis connection, required deps

Liveness failed → LB (or k8s) restarts the process
 Readiness failed → LB stops routing to this instance (no restart)

// Go health endpoint:

```
http.HandleFunc("/health/live", func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
})

http.HandleFunc("/health/ready", func(w http.ResponseWriter, r *http.Request) {
```

```

    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    if err := db.PingContext(ctx); err != nil {
        w.WriteHeader(http.StatusServiceUnavailable); return
    }
    w.WriteHeader(http.StatusOK)
})

```

Q45. ### Q66-Q75: More Load Balancing Topics

Difficulty:
Medium

Q	Topic
Q66	Rate limiting at the API gateway layer
Q67	Circuit breaker pattern with load balancers
Q68	Nginx configuration for reverse proxy and load balancing
Q69	AWS ALB vs NLB vs CLB: when to use each
Q70	Envoy proxy: architecture and use in service mesh
Q71	Connection draining: graceful traffic shifting
Q72	Canary deployments with load balancer traffic splitting
Q73	BGP Anycast load balancing for global services
Q74	GSLB (Global Server Load Balancing) across datacenters
Q75	WAF (Web Application Firewall) integration with LB

WebSockets, gRPC & Modern Protocols

Q46. What is WebSocket and how does it differ from HTTP?

Difficulty:
Medium

WebSocket: full-duplex bidirectional protocol over TCP
 Built on HTTP upgrade mechanism
 Single persistent TCP connection
 Low overhead: 2-byte framing (vs HTTP headers per request)

HTTP vs WebSocket:

HTTP: request/response (client must initiate every exchange)
 WebSocket: either side can send at any time (server push!)

Upgrade handshake:

Client → Server: HTTP GET with Upgrade: websocket header
 Server → Client: 101 Switching Protocols
 Now: WebSocket protocol (not HTTP anymore)

Use cases:

- Real-time chat (bidirectional messages)
- Live dashboards (server pushes updates)
- Collaborative editing (Google Docs, Figma)
- Gaming (low latency state updates)
- Financial data (live price feeds)

WebSocket vs SSE (Server-Sent Events):

- SSE: server → client only (unidirectional), HTTP-based, auto-reconnect
- WebSocket: bidirectional, separate protocol, manual reconnect
- SSE: simpler for server→client use cases (notifications, live scores)

```
// Go WebSocket server (gorilla/websocket):
upgrader := websocket.Upgrader{CheckOrigin: func(r *http.Request) bool { return true }}
conn, _ := upgrader.Upgrade(w, r, nil)
for {
    _, msg, err := conn.ReadMessage()
    if err != nil { break }
    conn.WriteMessage(websocket.TextMessage, processMsg(msg))
}
```

Q47. What is gRPC?

Difficulty:
Medium

gRPC: Google's RPC framework built on HTTP/2 + Protocol Buffers

Benefits over REST:

- Protocol Buffers: binary encoding (3-10× smaller than JSON)
- HTTP/2: multiplexing, header compression, streaming
- Type safety: .proto schema enforces types at compile time
- Code generation: client/server code in any language
- Bidirectional streaming: both sides can stream

Streaming modes:

- Unary: one request → one response (like REST)
- Server streaming: one request → many responses (live data)
- Client streaming: many requests → one response (upload)
- Bidirectional: many requests ↔ many responses (chat)

.proto definition:

```
service UserService {
    rpc GetUser (GetUserRequest) returns (User);
    rpc StreamUsers (Empty) returns (stream User);
    rpc BulkCreate (stream CreateUserRequest) returns (CreateResult);
    rpc Chat (stream ChatMessage) returns (stream ChatMessage);
}
```

Generated code:

- UserServiceClient interface in Go
- UserServiceServer interface to implement

Use cases:

- Internal microservices: faster, type-safe, streaming support
- Mobile apps: small payloads save bandwidth + battery
- Real-time: bidirectional streaming

Limitations:

- Not human-readable (binary)

No native browser support (needs grpc-web proxy)
Harder to debug (use grpcurl, Evans)

Q48. What is HTTP/2 server push?

Difficulty:
Medium

Server Push: server proactively sends resources without waiting for request
"I know you'll need index.css, here it is"

Use case: send critical resources before browser discovers and requests them

Traditional: HTML → parse → discover CSS → request CSS → CSS arrives

With Push: HTML (+ PUSH_PROMISE: index.css) → HTML + CSS arrive together

PUSH_PROMISE frame:

Server sends PUSH_PROMISE on stream N

Contains headers for pushed response (URL, content-type)

Client can accept or RST_STREAM to cancel push

Implementation:

Server: w.Header().Set("Link", "</style.css>; rel=preload; as=style")

Nginx: http2_push /style.css;

Client: can cache pushed resources for future requests

Limitations / controversy:

Browser must request same resource anyway (can't know if already cached)

Wasted bandwidth: push uncacheable resources

Complex implementation

Chrome removed support in 2022 (too much waste, 103 Early Hints better)

Modern alternative: 103 Early Hints

Server sends 103 status with Link headers early

Browser starts fetching hints while server processes request

Works with HTTP/1.1 and HTTP/2

Q49. What is gRPC streaming in Go?

Difficulty: **Hard**

```
// Server streaming: server sends many responses
func (s *Server) StreamOrders(req *pb.Empty, stream pb.OrderService_StreamOrdersServer) error {
    for _, order := range orders {
        if err := stream.Send(&pb.Order{Id: order.ID, Amount: order.Amount}); err != nil {
            return err
        }
    }
    return nil
}
```

```
// Client streaming: client sends many requests
func (s *Server) BulkCreate(stream pb.OrderService_BulkCreateServer) error {
    var count int32
    for {
        req, err := stream.Recv()
        if err == io.EOF {
            return stream.SendAndClose(&pb.CreateResult{Count: count})
        }
        if err != nil { return err }
        createOrder(req); count++
    }
}
```

```

}

// Bidirectional streaming: real-time chat
func (s *Server) Chat(stream pb.ChatService_ChatServer) error {
    for {
        msg, err := stream.Recv()
        if err == io.EOF { return nil }
        if err != nil { return err }
        broadcast(msg)
        stream.Send(&pb.Message{Content: "ack: " + msg.Content})
    }
}

// Client usage:
client := pb.NewOrderServiceClient(conn)
stream, _ := client.StreamOrders(ctx, &pb.Empty{})
for {
    order, err := stream.Recv()
    if err == io.EOF { break }
    if err != nil { log.Fatal(err) }
    fmt.Println(order)
}

```

Q50. What is gRPC vs REST trade-offs?

Difficulty:
Medium

	REST/JSON	gRPC/Protobuf
Protocol	HTTP/1.1 or 2	HTTP/2 (required)
Encoding	JSON (text)	Protobuf (binary)
Size	Larger	3-10× smaller
Speed	Slower	2-5× faster
Streaming	Limited	Native (4 modes)
Type safety	None (JSON)	Strong (.proto)
Browser	Native	Needs grpc-web
Schema	Optional (OAS)	Required
Code gen	Optional	Standard (protoc)
Debugging	Easy (curl)	Harder (grpcurl)
Human readable	Yes	No

When to use gRPC:

- Internal microservice communication (type safety + performance)
- Real-time streaming data
- Mobile apps (bandwidth constrained)
- Polyglot environments (generate in any language)

When to use REST:

- Public APIs (browser clients, third-party integrations)
- Human debugging needed
- Simple CRUD operations
- CDN caching of GET responses

Typical architecture:

- External API (browser/mobile/partners): REST + JSON
- Internal service-to-service: gRPC
- API Gateway: transcodes REST → gRPC (grpc-gateway)

Q51. ### Q81-Q88: More Protocol Topics**Difficulty:**
Medium

Q	Topic
Q81	SSE (Server-Sent Events): implementation and use cases
Q82	Long polling vs short polling vs WebSocket
Q83	Protocol Buffers: schema design and evolution
Q84	gRPC interceptors for authentication and tracing
Q85	gRPC health checking (grpc.health.v1.Health)
Q86	WebSocket connection management at scale
Q87	gRPC load balancing: client-side vs proxy
Q88	QUIC and HTTP/3 implementation in Go

Go Networking Patterns

Q52. How do you build an HTTP server in Go with proper timeouts?**Difficulty:** Easy

```

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/users", handleUsers)

    server := &http.Server{
        Addr:    ":8080",
        Handler: mux,

        // ALWAYS set these to prevent resource exhaustion
        ReadTimeout:  10 * time.Second, // time to read request (including body)
        WriteTimeout: 30 * time.Second, // time to write response
        IdleTimeout:  120 * time.Second, // keep-alive connection idle timeout
        ReadHeaderTimeout: 5 * time.Second, // time to read request headers

        MaxHeaderBytes: 1 << 20, // 1MB max header size
    }

    // Graceful shutdown
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)

    go func() {
        if err := server.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
            log.Fatalf("server error: %v", err)
        }
    }()

    <-quit
    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()
    if err := server.Shutdown(ctx); err != nil {

```

```

        log.Printf("forced shutdown: %v", err)
    }
}

```

Q53. How do you make resilient HTTP client calls in Go?

Difficulty:
Medium

```

// Reusable client with proper configuration
var httpClient = &http.Client{
    Timeout: 30 * time.Second,
    Transport: &http.Transport{
        MaxIdleConns:        100,
        MaxIdleConnsPerHost: 10,
        MaxConnsPerHost:     100,
        IdleConnTimeout:     90 * time.Second,
        TLSHandshakeTimeout: 10 * time.Second,
        ResponseHeaderTimeout: 10 * time.Second,
        ExpectContinueTimeout: 1 * time.Second,
        // Enable HTTP/2
        ForceAttemptHTTP2: true,
    },
}

// Call with retry and context
func callWithRetry(ctx context.Context, url string) ([]byte, error) {
    backoff := 100 * time.Millisecond
    for attempt := 0; attempt < 3; attempt++ {
        req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
        req.Header.Set("X-Request-ID", requestID(ctx))

        resp, err := httpClient.Do(req)
        if err != nil {
            if !isRetryable(err) { return nil, err }
            time.Sleep(backoff); backoff *= 2
            continue
        }
        defer resp.Body.Close()

        if resp.StatusCode >= 500 {
            time.Sleep(backoff); backoff *= 2; continue
        }

        body, err := io.ReadAll(io.LimitReader(resp.Body, 10<<20)) // max 10MB
        return body, err
    }
    return nil, errors.New("max retries exceeded")
}

func isRetryable(err error) bool {
    var urlErr *url.Error
    return errors.As(err, &urlErr) && (urlErr.Timeout() || urlErr.Temporary())
}

```

Q54. How do you implement graceful shutdown in Go?

Difficulty:
Medium

```

func main() {

```

```

srv := &http.Server{Addr: ":8080", Handler: buildRouter()}

// Start server
go func() {
    log.Println("server starting on :8080")
    if err := srv.ListenAndServe(); !errors.Is(err, http.ErrServerClosed) {
        log.Fatalf("server error: %v", err)
    }
}()

// Wait for interrupt signal
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
sig := <-quit
log.Printf("received signal: %v, shutting down gracefully...", sig)

// Give in-flight requests 30 seconds to complete
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

// Shutdown: stop accepting new connections, wait for active to complete
if err := srv.Shutdown(ctx); err != nil {
    log.Printf("server forced to shutdown: %v", err)
}

// Close other resources (DB, Redis, Kafka)
log.Println("closing database connections...")
dbPool.Close()
redisClient.Close()

log.Println("server exited cleanly")
}

```

Q55. How do you build a WebSocket server in Go?

Difficulty:
Medium

```

import "github.com/gorilla/websocket"

var upgrader = websocket.Upgrader{
    ReadBufferSize: 1024,
    WriteBufferSize: 1024,
    CheckOrigin:    func(r *http.Request) bool { return true },
}

type Hub struct {
    clients    map[*websocket.Conn]bool
    broadcast  chan []byte
    register   chan *websocket.Conn
    unregister chan *websocket.Conn
    mu         sync.RWMutex
}

func (h *Hub) Run() {
    for {
        select {
        case conn := <-h.register:
            h.mu.Lock()

```

```

        h.clients[conn] = true
        h.mu.Unlock()
    case conn := <-h.unregister:
        h.mu.Lock()
        delete(h.clients, conn)
        h.mu.Unlock()
        conn.Close()
    case msg := <-h.broadcast:
        h.mu.RLock()
        for conn := range h.clients {
            conn.WriteMessage(websocket.TextMessage, msg)
        }
        h.mu.RUnlock()
    }
}

func handleWS(hub *Hub, w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil { return }
    hub.register <- conn

    go func() {
        defer func() { hub.unregister <- conn }()
        conn.SetReadDeadline(time.Now().Add(60 * time.Second))
        conn.SetPongHandler(func(string) error {
            conn.SetReadDeadline(time.Now().Add(60 * time.Second))
            return nil
        })
        for {
            _, msg, err := conn.ReadMessage()
            if err != nil { break }
            hub.broadcast <- msg
        }
    }()
}

```

Q56. How do you build a gRPC server in Go?

Difficulty:
Medium

```

import (
    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials"
    "google.golang.org/grpc/keepalive"
)

// Implement generated interface
type userServer struct {
    pb.UnimplementedUserServiceServer
    repo UserRepository
}

func (s *userServer) GetUser(ctx context.Context, req *pb.GetUserRequest) (*pb.User, error) {
    user, err := s.repo.GetByID(ctx, req.Id)
    if err != nil {
        return nil, status.Errorf(codes.NotFound, "user %d not found", req.Id)
    }
}

```

```

    return &pb.User{Id: user.ID, Name: user.Name, Email: user.Email}, nil
}

func main() {
    lis, _ := net.Listen("tcp", ":50051")

    creds, _ := credentials.NewServerTLSFromFile("cert.pem", "key.pem")

    srv := grpc.NewServer(
        grpc.Creds(creds),
        grpc.KeepaliveParams(keepalive.ServerParameters{
            MaxConnectionIdle: 15 * time.Second,
            Time:               5 * time.Second,
            Timeout:            1 * time.Second,
        }),
        grpc.ChainUnaryInterceptor(
            authInterceptor,
            tracingInterceptor,
            loggingInterceptor,
        ),
        grpc.MaxRecvMsgSize(10 * 1024 * 1024), // 10MB
    )

    pb.RegisterUserServiceServer(srv, &userServer{repo: repo})

    // Register reflection for grpcurl debugging
    reflection.Register(srv)

    srv.Serve(lis)
}

```

Q57. What are Go HTTP middleware patterns?

Difficulty:
Medium

```

// Middleware type: wraps http.Handler
type Middleware func(http.Handler) http.Handler

// Logging middleware
func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        rec := &statusRecorder{ResponseWriter: w, status: 200}
        next.ServeHTTP(rec, r)
        log.Printf("%s %s %d %v", r.Method, r.URL.Path, rec.status, time.Since(start))
    })
}

// Auth middleware
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        userID, err := validateToken(token)
        if err != nil { http.Error(w, "unauthorized", 401); return }
        ctx := context.WithValue(r.Context(), ctxKeyUserID, userID)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

```

```
// Chain middlewares
func Chain(h http.Handler, middlewares ...Middleware) http.Handler {
    for i := len(middlewares) - 1; i >= 0; i-- {
        h = middlewares[i](h)
    }
    return h
}

// Usage:
mux := http.NewServeMux()
mux.HandleFunc("/users", handleUsers)
handler := Chain(mux, LoggingMiddleware, AuthMiddleware, CorsMiddleware)
http.ListenAndServe(":8080", handler)
```

Q58. ### Q95-Q100: More Go Networking Topics

Difficulty:
Medium

Q	Topic
Q95	Context propagation and cancellation in Go HTTP handlers
Q96	HTTP/2 in Go: enabling and benefits
Q97	gRPC client with interceptors (retry, tracing, auth)
Q98	DNS client in Go: custom resolver, caching
Q99	TCP server in Go: raw connection handling
Q100	Network debugging in Go: net/http/httptrace, pprof

Master these 100 questions and you'll handle any networking interview at SDE2 level. Key areas: TCP/IP (handshake, flow control, keepalive), HTTP versions (1.1 vs 2 vs 3), DNS resolution, TLS/mTLS, load balancing algorithms, gRPC vs REST, and Go networking patterns (server, client, WebSocket, graceful shutdown). ■

Section 3 — Transport Layer & Protocols (Q59–Q100)

Q59. What is TCP three-way handshake?

Difficulty:
Medium

TCP Connection Establishment (3-way handshake):

Client	Server
---SYN (seq=x)----->	Client sends SYN, random seq number
<--SYN-ACK (seq=y,ack=x+1)--	Server responds with SYN+ACK
---ACK (ack=y+1)----->	Client acknowledges → connection ESTABLISHED

SYN = synchronize sequence numbers

ACK = acknowledge

States:

Client: CLOSED → SYN_SENT → ESTABLISHED
Server: LISTEN → SYN_RECEIVED → ESTABLISHED

Time: 1.5 RTT (Round Trip Time) to establish connection

TCP Four-way handshake (termination):

FIN → ACK → FIN → ACK
TIME_WAIT state: 2×MSL (Maximum Segment Lifetime, usually 60s)
Ensures last ACK received by peer

SYN flood attack: send many SYN, never complete handshake

Defense: SYN cookies (server doesn't allocate state until ACK)

Q60. What is TCP vs UDP?**Difficulty: Easy****TCP (Transmission Control Protocol):**

Connection-oriented (3-way handshake)
Reliable delivery (ACK, retransmit on loss)
Ordered delivery (sequence numbers)
Flow control (receiver window)
Congestion control (slow start, AIMD)
Error detection (checksum)
Full-duplex

Use: HTTP/HTTPS, email (SMTP), file transfer (FTP), SSH, databases

UDP (User Datagram Protocol):

Connectionless
Unreliable (no ACK, no retransmit)
No ordering guarantee
No flow/congestion control
Lightweight, low overhead
Best effort delivery

Use: DNS, video streaming, gaming, VoIP, DHCP, QUIC

Performance comparison:

TCP header: 20-60 bytes
UDP header: 8 bytes (src port, dst port, length, checksum)

TCP latency: +1.5 RTT for handshake

UDP latency: immediate first packet

When to choose UDP:

- Latency more important than reliability (gaming, VoIP)
- App handles retransmission itself (QUIC)
- Small queries (DNS: 1 request, 1 response)
- Broadcast/multicast

Q61. What is TCP flow control and congestion control?**Difficulty: Hard**

Flow Control: prevent fast sender from overwhelming slow receiver

Receive Window (rwnd): advertised by receiver in every ACK

Sender can have at most rwnd bytes unacknowledged

If rwnd=0 → sender pauses (Zero Window)

Congestion Control: prevent overwhelming the network

Algorithms: Slow Start, Congestion Avoidance, Fast Retransmit, Fast Recovery

Slow Start:

cwnd (congestion window) starts at 1 MSS

Doubles each RTT until ssthresh (slow start threshold)

Congestion Avoidance:

Above ssthresh: increment cwnd by 1 MSS per RTT (linear)

Loss detection:

Triple duplicate ACK → Fast Retransmit (faster than timeout)

ssthresh = cwnd/2, cwnd = ssthresh

Timeout: ssthresh = cwnd/2, cwnd = 1 (worst case)

Effective window = min(rwnd, cwnd)

Bandwidth-Delay Product: max bytes in flight = bandwidth × RTT

Modern algorithms:

CUBIC (Linux default): cubic function for cwnd growth

BBR (Google): model-based, optimizes for BDP

QUIC: UDP-based, own congestion control

Q62. What is HTTP/1.1 vs HTTP/2 vs HTTP/3?

Difficulty: Hard

HTTP/1.1:

Text-based protocol

One request at a time per connection (HOL blocking)

Workaround: multiple connections (6 per origin typically)

Keep-Alive: reuse connection for multiple requests

Pipelining: send multiple requests without waiting (broken in practice)

HTTP/2:

Binary framing layer

Multiplexing: multiple streams on single TCP connection

Header compression (HPACK)

Server push (deprecated/limited)

Stream prioritization

Still has TCP HOL blocking (one lost packet blocks all streams)

HTTP/3 (QUIC):

Built on UDP (not TCP)

No TCP HOL blocking (each stream independent)

0-RTT connection resumption

Connection migration (change IP without reconnecting)

Built-in TLS 1.3

Better for mobile, lossy networks

Performance comparison:

HTTP/1.1: N connections × serial requests

HTTP/2: 1 connection × parallel streams (better on good network)

HTTP/3: 1 connection × parallel streams (better on lossy network)

Header comparison:

HTTP/1.1: ~800 bytes per request (repetitive headers)

HTTP/2: compressed headers (HPACK), ~50-100 bytes

HTTP/3: compressed headers (QPACK)

Q63. What is TLS/SSL handshake?**Difficulty: Hard**

TLS 1.3 Handshake (1-RTT):

Client	Server
--ClientHello (supported algos)--->	
+ KeyShare (ECDHE public key)	
<-ServerHello (chosen algo)-----	
+ KeyShare (server ECDHE key)	
+ Certificate	
+ CertificateVerify	
+ Finished (encrypted)	
--Finished (encrypted)----->	
=== Application Data ===	

TLS 1.3 improvements over TLS 1.2:

1-RTT vs 2-RTT

0-RTT resumption (session tickets, replays are a concern)

Removed weak algorithms (RSA key exchange, SHA-1, MD5, CBC)

Always forward secrecy (ECDHE required)

Encrypted handshake (certificate hidden from passive observer)

Key exchange: ECDHE (Elliptic Curve Diffie-Hellman Ephemeral)

Provides forward secrecy: compromising private key doesn't decrypt past sessions (new key per session)

Certificates: X.509

CA (Certificate Authority) chain of trust

Let's Encrypt: free, automated certificates

mTLS: client also presents certificate (mutual authentication)

Q64. What are HTTP status codes?**Difficulty: Easy**

1xx Information:

100 Continue	- server received headers, send body
101 Switching	- upgrading to WebSocket

2xx Success:

200 OK	- standard success
201 Created	- resource created (POST)
202 Accepted	- async processing started
204 No Content	- success, no body (DELETE, PUT)
206 Partial Content	- range request (streaming)

3xx Redirection:

301 Moved Permanently	- permanent redirect (update bookmarks)
302 Found	- temporary redirect
304 Not Modified	- ETag/Last-Modified match (use cache)
307 Temporary Redirect	- preserve method (POST stays POST)
308 Permanent Redirect	- like 307 but permanent

4xx Client Error:

```

400 Bad Request      - invalid request body/params
401 Unauthorized     - missing/invalid authentication
403 Forbidden        - authenticated but not authorized
404 Not Found        - resource doesn't exist
405 Method Not Allowed
409 Conflict         - state conflict (optimistic lock)
410 Gone            - resource deleted permanently
422 Unprocessable    - validation error
429 Too Many Requests - rate limited

```

5xx Server Error:

```

500 Internal Server Error - unhandled exception
502 Bad Gateway          - upstream error (nginx → app)
503 Service Unavailable - overload/maintenance
504 Gateway Timeout      - upstream timeout

```

Q65. What is REST API design principles?

Difficulty: Easy

REST (Representational State Transfer) principles:

1. Stateless: each request contains all info needed
2. Client-Server: separation of concerns
3. Uniform Interface: consistent URI patterns
4. Cacheable: responses can be cached
5. Layered System: proxies, load balancers transparent

URI conventions:

Resources (nouns, not verbs):

```

GET    /users          - list users
POST   /users          - create user
GET    /users/{id}     - get specific user
PUT    /users/{id}     - replace user
PATCH /users/{id}     - partial update
DELETE /users/{id}     - delete user

```

Nested resources:

```

GET /users/{id}/orders - user's orders
POST /users/{id}/orders - create order for user
GET /users/{id}/orders/{oid} - specific order

```

Query parameters:

```

GET /users?page=1&limit=20&sort=name&order=asc
GET /users?status=active&role=admin

```

Versioning:

```

/api/v1/users (URL versioning - most common)
Accept: application/vnd.api.v1+json (header versioning)
api.v1.example.com (subdomain versioning)

```

Response format:

JSON with consistent structure

```

{
  "data": { ... },
  "meta": { "total": 100, "page": 1 },
  "errors": [{ "code": "INVALID_EMAIL", "field": "email" }]
}

```

Q66. What is gRPC vs REST?**Difficulty:**
Medium**REST:**

Protocol: HTTP/1.1 or HTTP/2
 Format: JSON (human-readable, larger)
 Schema: OpenAPI/Swagger (optional)
 Streaming: polling or SSE (limited)
 Browser support: native
 Code generation: optional
 Use: public APIs, browser clients, simple CRUD

gRPC:

Protocol: HTTP/2 (always)
 Format: Protocol Buffers (binary, smaller, faster)
 Schema: .proto files (required, strongly typed)
 Streaming: unary, server-streaming, client-streaming, bidirectional
 Browser support: grpc-web (limited, needs proxy)
 Code generation: required (stubs in many languages)
 Use: microservices, internal APIs, streaming, polyglot systems

Performance:

gRPC: ~5-10x faster serialization, lower bandwidth (binary)
 REST: ~2-3x slower, but simpler

Proto vs JSON:

JSON: {"name":"Alice","age":30} → 24 bytes
 Protobuf: same data → ~10 bytes (binary)

gRPC call types:

Unary: 1 request → 1 response (like REST)
 Server streaming: 1 request → N responses (live data)
 Client streaming: N requests → 1 response (file upload)
 Bidirectional: N requests ↔ N responses (chat, real-time)

When to choose gRPC:

Internal microservices (performance, type safety)
 Streaming data (real-time feeds)
 Polyglot (auto-generate clients in any language)
 Strong API contracts required

Q67. What is WebSocket protocol?**Difficulty:**
Medium

WebSocket: full-duplex communication over single TCP connection

Upgrade from HTTP: uses HTTP for handshake, then switches

Handshake:

Client → Server:

GET /chat HTTP/1.1
 Upgrade: websocket
 Connection: Upgrade
 Sec-WebSocket-Key: dGhliHNhbXBsZSBub25jZQ==
 Sec-WebSocket-Version: 13

Server → Client:

HTTP/1.1 101 Switching Protocols

```
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

After handshake: raw frames (not HTTP) over same TCP connection

Frame format:

FIN + opcode + mask + payload length + masking key + payload

Opcodes:

```
0x1: text frame (UTF-8)
0x2: binary frame
0x8: close
0x9: ping (keepalive)
0xA: pong (response to ping)
```

Use cases: real-time dashboards, chat, collaborative editing, gaming

vs SSE:

```
SSE: server→client only, HTTP, auto-reconnect, simpler
WebSocket: bidirectional, custom protocol, more complex
Choose SSE for: notifications, feeds (unidirectional)
Choose WebSocket for: chat, gaming (bidirectional)
```

Q68. What is DNS resolution process?

Difficulty:
Medium

DNS Resolution (for google.com):

Browser DNS cache → OS DNS cache → Local resolver (/etc/resolv.conf)

↓ (cache miss)

Recursive resolver (ISP or 8.8.8.8):

↓ (cache miss)

Root nameserver (13 clusters, type "."):

Returns: "com" TLD nameserver addresses

↓

TLD nameserver (.com):

Returns: google.com authoritative nameserver addresses

↓

Authoritative nameserver (ns1.google.com):

Returns: 142.250.80.46 (A record)

↓

Recursive resolver caches result (per TTL)

↓

Client gets IP, makes HTTP connection

Record types:

```
A:      hostname → IPv4 (142.250.80.46)
AAAA:   hostname → IPv6
CNAME:  alias → another hostname (www → myapp.loadbalancer.com)
MX:     mail exchange (smtp.google.com priority 10)
NS:     nameserver for domain
TXT:    arbitrary text (SPF, DKIM, domain verification)
SOA:    Start of Authority (serial, refresh, retry, expire, TTL)
PTR:    reverse DNS (IP → hostname)
SRV:    service location (_http._tcp.example.com)
```

TTL: cache duration (low TTL = quick propagation, high TTL = fewer queries)

Q69. What is load balancing algorithms?**Difficulty:**
Medium

Load Balancing Algorithms:

1. Round Robin:

Request 1 → Server A

Request 2 → Server B

Request 3 → Server C

Request 4 → Server A ...

Simple, equal distribution, doesn't consider server load

2. Weighted Round Robin:

Server A (weight=3): 3 out of every 5 requests

Server B (weight=2): 2 out of every 5 requests

Use when: servers have different capacities

3. Least Connections:

Route to server with fewest active connections

Good for: long-lived connections, unequal request duration

4. Weighted Least Connections:

Combines weight + active connections

Best general-purpose algorithm

5. IP Hash:

hash(client_IP) % num_servers → same server always

Session affinity (sticky sessions)

Problem: poor distribution if few IPs (NAT)

6. Least Response Time:

Route to server with lowest latency + fewest connections

Use when: response time is critical metric

7. Random:

Simple, works well with many servers (law of large numbers)

8. Consistent Hashing:

Used for: caching (same key → same cache node)

Minimizes remapping when nodes added/removed

Layer 4 (transport): based on IP+port (faster, no TLS termination)

Layer 7 (application): based on HTTP headers, URL, cookies (smarter routing)

Q70. What is NAT (Network Address Translation)?**Difficulty:**
Medium

NAT: translate private IPs to public IPs for internet access

Types:

SNAT (Source NAT): change source IP (most common for outbound)

DNAT (Destination NAT): change destination IP (port forwarding)

Masquerade: SNAT with dynamic public IP (home router)

PAT (Port Address Translation): many-to-one (most common home/office)

Example (PAT/Overloading):

```
192.168.1.10:12345 → router → 1.2.3.4:45678 → internet
192.168.1.11:12345 → router → 1.2.3.4:45679 → internet
```

```
NAT table: (public_port → private_ip:private_port)
45678 → 192.168.1.10:12345
45679 → 192.168.1.11:12345
```

Problems with NAT:

```
P2P connections (both behind NAT → can't connect)
WebRTC requires STUN/TURN servers for NAT traversal
Connection tracking state (firewall must track)
End-to-end connectivity broken
```

In cloud (AWS):

```
Internet Gateway: provides public IP for VPC instances
NAT Gateway: allows private subnet instances to reach internet
ELB DNAT: translates incoming traffic to backend IPs
```

Docker uses iptables NAT rules:

```
iptables -t nat -L DOCKER # view Docker's NAT rules
```

Q71. What is OSI vs TCP/IP model?

Difficulty: Easy

OSI Model (7 layers):	TCP/IP Model (4 layers):
7. Application ██████████	
6. Presentation ██████████	Application (HTTP, gRPC, DNS, SMTP)
5. Session ██████████	
4. Transport ██████████	Transport (TCP, UDP)
3. Network ██████████	Internet (IP, ICMP, ARP)
2. Data Link ██████████	
1. Physical ██████████	Network Access (Ethernet, WiFi)

Each layer encapsulates the layer above:

```
[Ethernet header [IP header [TCP header [HTTP data]]]]
```

Layer responsibilities:

```
7 Application: HTTP, FTP, DNS, SMTP, gRPC
6 Presentation: encoding, encryption, compression
5 Session: session establishment/termination
4 Transport: TCP (reliable), UDP (best effort), port numbers
3 Network: IP routing, packet forwarding, subnets
2 Data Link: MAC addresses, frames, error detection, switches
1 Physical: bits → signals, cables, WiFi, fiber
```

Protocols per layer:

```
App: HTTP, HTTPS, WebSocket, gRPC, DNS, DHCP, SMTP
Transport: TCP, UDP, SCTP
Network: IPv4, IPv6, ICMP, OSPF, BGP
Link: Ethernet (IEEE 802.3), WiFi (IEEE 802.11), ARP
```

Q72. What is subnet and CIDR notation?

Difficulty: Medium

CIDR (Classless Inter-Domain Routing):

```
IP/prefix_length notation
192.168.1.0/24
```

```
/24 means: 24 bits for network, 8 bits for hosts
Network:   192.168.1.0
Netmask:   255.255.255.0
Hosts:     192.168.1.1 - 192.168.1.254 (254 usable)
Broadcast: 192.168.1.255
```

Common CIDRs:

```
/32: single host (1 address)
/30: 4 addresses, 2 usable (point-to-point links)
/28: 16 addresses, 14 usable
/24: 256 addresses, 254 usable (typical LAN)
/16: 65536 addresses (large org, VPC)
/8:  16M addresses (class A)
/0:  all addresses (default route)
```

Private address ranges (RFC 1918):

```
10.0.0.0/8      (10.x.x.x - large corporate)
172.16.0.0/12   (172.16-31.x.x)
192.168.0.0/16  (192.168.x.x - home/small office)
```

AWS VPC example:

```
VPC:           10.0.0.0/16
Public subnet:  10.0.1.0/24 (internet-facing)
Private subnet: 10.0.2.0/24 (app servers)
DB subnet:      10.0.3.0/24 (databases)
```

Subnetting: divide network into smaller networks

```
10.0.0.0/16 → 10.0.0.0/24, 10.0.1.0/24, 10.0.2.0/24 ...
```

Q73. What is BGP (Border Gateway Protocol)?

Difficulty: Hard

BGP: the routing protocol of the internet

AS (Autonomous System): network under single administrative control

Each ISP, cloud provider, large org has one or more AS numbers

AWS AS: 16509, Google AS: 15169

BGP:

External BGP (eBGP): between different ASes

Internal BGP (iBGP): within same AS

Route advertisement:

AS1 → AS2 → AS3 → AS4 (destination)

AS4 advertises 1.2.3.0/24 to AS3

AS3 prepends its AS number, forwards to AS2

Path: AS1 knows route is AS2→AS3→AS4 for 1.2.3.0/24

Route selection (simplified):

Shortest AS path

Lowest MED (Multi-Exit Discriminator)

Prefer eBGP over iBGP

Lowest IGP cost to next hop

BGP in cloud:

AWS Direct Connect: BGP between on-prem router and AWS

VPN: BGP for dynamic routing over IPsec tunnel

AWS Transit Gateway: BGP for multi-VPC routing

BGP hijacking: AS advertises routes it doesn't own
 Famous incident: Pakistan Telecom hijacked YouTube (2008)
 Defense: RPKI (Route Origin Authorization), BGPsec

Q74. What is HTTPS and certificate chain?

Difficulty:
Medium

HTTPS = HTTP + TLS

Certificate chain:

```
Root CA (trusted, pre-installed in OS/browser)
  ■■■ Intermediate CA (cross-signed by Root CA)
    ■■■ Server Certificate (issued to example.com)
```

Certificate fields:

```
Subject:   CN=example.com, O=My Company
Issuer:    CN=Let's Encrypt R3
Valid:     Not Before, Not After
Public Key: RSA 2048 or EC P-256
SANs:      Subject Alternative Names (*.example.com, example.com)
Extensions: Key Usage, Extended Key Usage
```

Verification:

1. Server presents certificate
2. Client checks: cert signed by trusted CA (chain to root)
3. Client checks: hostname matches CN/SAN
4. Client checks: certificate not expired
5. Client checks: certificate not revoked (CRL/OCSP)

Certificate pinning:

```
App pins to specific cert/public key
Prevents MITM even with rogue CA
Downside: cert rotation requires app update
```

HSTS (HTTP Strict Transport Security):

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Browser remembers: always use HTTPS for this domain
```

Certificate Transparency:

```
All certs logged to public CT logs
Prevents secret cert issuance
```

Q75. What is HTTP caching?

Difficulty:
Medium

HTTP Caching headers:

Cache-Control (most important):

```
Cache-Control: max-age=3600           # cache for 1 hour
Cache-Control: no-cache               # revalidate before using cached copy
Cache-Control: no-store               # don't cache at all
Cache-Control: private                # only client can cache (not CDN)
Cache-Control: public                 # CDN can cache
Cache-Control: s-maxage=86400         # CDN max age (overrides max-age)
Cache-Control: must-revalidate        # don't serve stale even in errors
Cache-Control: stale-while-revalidate=60 # serve stale, refresh in background
```

ETag (validation):

```
Server sends: ETag: "v1-abc123"  
Client caches response with ETag  
Next request: If-None-Match: "v1-abc123"  
If unchanged: 304 Not Modified (no body)  
If changed: 200 OK with new body + new ETag
```

Last-Modified:

```
Server: Last-Modified: Tue, 15 Jan 2024 10:00:00 GMT  
Client: If-Modified-Since: Tue, 15 Jan 2024 10:00:00 GMT  
If unchanged: 304 Not Modified
```

CDN caching strategy:

```
Static assets: Cache-Control: public, max-age=31536000, immutable  
API responses: Cache-Control: private, no-store  
Or: Cache-Control: public, s-maxage=60 (CDN for 60s)  
HTML: Cache-Control: no-cache (always revalidate)
```

Q76. What is rate limiting algorithms?

Difficulty: Hard

1. Fixed Window:

```
Count requests in current time window (e.g., 1 minute)  
Reset at window boundary  
Problem: burst at window boundary (2x limit in 1 second)  
Simple to implement
```

2. Sliding Window Log:

```
Store timestamp of each request  
Count requests in last N seconds  
Accurate but high memory (store every timestamp)
```

3. Sliding Window Counter:

```
Approximate sliding window using two fixed windows  
current_window_count × (time_remaining/window_size) + prev_window_count  
Memory efficient, approximate
```

4. Token Bucket:

```
Bucket holds up to N tokens  
Tokens refilled at fixed rate (e.g., 10/sec)  
Request costs 1 token; rejected if empty  
Allows bursting (accumulate tokens when idle)
```

5. Leaky Bucket:

```
Requests enter queue; processed at fixed rate (drains like leak)  
Smooths traffic, no bursting  
Good for: output rate limiting
```

Comparison:

```
Fixed window: simple, burst problem  
Sliding window: accurate, memory intensive  
Token bucket: allows burst, most common for APIs  
Leaky bucket: smoothing, good for network shaping
```

Implementation:

```
Centralized: Redis (atomic INCR + EXPIRE)  
Distributed: Redis Cluster, consistent hashing per user  
Libraries: golang.org/x/time/rate (token bucket), redis rate limiting
```

Q77. What is CDN (Content Delivery Network)?**Difficulty:**
Medium

CDN: geographically distributed servers cache content close to users

How it works:

1. User requests `example.com/image.jpg`
2. DNS resolves to nearest CDN edge server (anycast routing)
3. Edge serves from cache (HIT) → fast response
4. Cache MISS → edge fetches from origin, caches it
5. Subsequent requests: served from edge cache

Cache hierarchy:

Edge nodes (PoPs) → Regional nodes → Origin

Key concepts:

TTL: how long CDN caches content
Cache invalidation: purge when content changes
Cache key: URL + headers (Vary header)
Stale-while-revalidate: serve stale, refresh async
Cache warming: pre-populate before traffic spikes

CDN benefits:

Reduced latency (closer to user)
Reduced origin server load
DDoS protection (absorb attack at edge)
SSL termination (at edge, origin can use HTTP internally)
Compression (gzip/brotli)

CDN providers:

Cloudflare, AWS CloudFront, Fastly, Akamai, Azure CDN

What to cache:

- ✓ Static assets (JS, CSS, images) – immutable URLs + long TTL
- ✓ API responses (public, non-personalized) – short TTL
- ✗ Personal data, auth tokens, session-specific content
- ✗ Real-time data (stock prices, live feeds)

Q78. What is TCP keepalive?**Difficulty:**
Medium

TCP keepalive: detect dead connections

Problem: silent disconnect

Client connects to server
Network device (NAT/firewall) times out the mapping
Client thinks connection is alive, server thinks alive
Next request → timeout (no error until send)

TCP Keepalive:

After idle time (default 2 hours), send keepalive probes
If no response after N probes → mark connection dead

Linux settings:

```
net.ipv4.tcp_keepalive_time=7200    # idle before probes (2 hours)
net.ipv4.tcp_keepalive_intvl=75     # interval between probes
net.ipv4.tcp_keepalive_probes=9     # probes before declaring dead
```

Application-level keepalive (better control):

- HTTP keep-alive: reuse TCP connection for multiple requests
- WebSocket ping/pong (every 30s)
- gRPC keepalive:
 - grpc.keepalive_time_ms: 10000
 - grpc.keepalive_timeout_ms: 5000

Go HTTP client:

- Transport.IdleConnTimeout: close idle connections
- Transport.MaxIdleConns: pool size
- Transport.DisableKeepAlives: disable (each request new conn)

Q79. What is VPN and tunneling?

Difficulty:
Medium

VPN (Virtual Private Network): encrypt traffic over public internet

Types:

- Site-to-Site VPN: connect two networks (office ↔ cloud)
- Remote Access VPN: user → corporate network

Protocols:

- IPsec: L3, high performance, complex config
 - IKEv2: key exchange
 - ESP: encryption, AH: authentication

OpenVPN: TLS-based, flexible, widely compatible

WireGuard: modern, fast, minimal code (4000 lines vs 400K IPsec)
Uses: ChaCha20, Poly1305, Curve25519, BLAKE2s

Tunneling:

SSH tunneling:

Local forward: `ssh -L 5432:db.internal:5432 bastion.example.com`
→ connects localhost:5432 through SSH to remote db:5432

Dynamic (SOCKS proxy): `ssh -D 1080 bastion.example.com`

HTTP tunneling (CONNECT):

`CONNECT proxy.example.com:8080 HTTP/1.1`
→ TCP tunnel through HTTP proxy

In AWS:

- AWS VPN: IPsec tunnel between on-prem and VPC
- AWS Direct Connect: dedicated physical link (better performance)
- VPC Peering: connect two VPCs (no VPN, AWS backbone)
- Transit Gateway: hub-and-spoke VPC connectivity

Q80. What is network security: firewall, WAF, IDS/IPS?

Difficulty:
Medium

Firewall: filter traffic based on rules

- Packet filter: IP/port rules (iptables, nftables)
- Stateful: tracks connection state (allows return traffic)
- Application-aware: understands protocols (HTTP, DNS)

iptables example:

```
Allow established connections:
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
Allow HTTP/HTTPS:
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
Block all else:
iptables -P INPUT DROP
```

WAF (Web Application Firewall):

- Layer 7 (HTTP): inspect request body, headers, URLs
- Protect against: SQLi, XSS, CSRF, SSRF
- Rules: OWASP ModSecurity Core Rule Set
- AWS: AWS WAF, Cloudflare WAF
- Block: known attack patterns, IP reputation, rate limiting

IDS (Intrusion Detection System):

- Detect attacks, generate alerts
- Signature-based: match known attack patterns
- Anomaly-based: detect deviations from baseline

IPS (Intrusion Prevention System):

- IDS + automatic blocking

Security groups (AWS):

- Stateful firewall at instance level
- Default: deny all inbound, allow all outbound

NACLs (Network ACLs, AWS):

- Stateless firewall at subnet level
- Must explicitly allow return traffic

Q81. What is Nginx vs HAProxy?

Difficulty:
Medium

Nginx:

- Web server + reverse proxy + load balancer
- Event-driven, async (handles 10K+ connections)
- Excellent for: static files, SSL termination, HTTP load balancing

Features: HTTP/2, gzip, caching, rate limiting, auth, rewrite rules

Config:

```
upstream backend {
    server app1:8080;
    server app2:8080;
    keepalive 32;
}
server {
    listen 443 ssl http2;
    location /api/ { proxy_pass http://backend; }
}
```

HAProxy:

- Dedicated load balancer, extremely performant
- Both Layer 4 (TCP) and Layer 7 (HTTP) load balancing
- Features: health checks, ACLs, stats dashboard, queue
- Better for: TCP load balancing, advanced routing, statistics

Config:

```

frontend http_front
  bind *:80
  default_backend http_back
backend http_back
  balance roundrobin
  server app1 192.168.1.1:8080 check
  server app2 192.168.1.2:8080 check

```

vs AWS ALB/NLB:

ALB: Layer 7, path-based routing, AWS-managed

NLB: Layer 4, ultra-high throughput, preserves client IP

ELB: classic, legacy, avoid

Q82. What is IPv4 vs IPv6?

Difficulty: Easy

IPv4:

32-bit addresses: 4 billion total (4.3×10^9)

Exhausted (IANA ran out in 2011)

Format: 192.168.1.1 (dotted decimal)

Private ranges: 10.x.x.x, 172.16-31.x.x, 192.168.x.x

Header: 20-60 bytes

IPv6:

128-bit addresses: 340 undecillion (3.4×10^{38})

Format: 2001:0db8:85a3::8a2e:0370:7334 (hex, colon-separated)

:: = consecutive zeros

Loopback: ::1 (like 127.0.0.1)

Link-local: fe80::/10 (auto-configured per interface)

Global unicast: 2000::/3

No NAT needed (each device gets public IP)

Auto-configuration: SLAAC (no DHCP required)

Better: built-in IPsec, simpler header, no fragmentation by routers

Dual-stack: run IPv4 and IPv6 simultaneously

Transition:

6to4, Teredo: tunneling IPv6 over IPv4

NAT64/DNS64: translate between v4 and v6

In Go:

```

net.Listen("tcp", ":8080")           # listens on both IPv4 and IPv6
net.Listen("tcp4", ":8080")          # IPv4 only
net.Listen("tcp6", ":8080")          # IPv6 only

```

Q83. What is service mesh (Istio, Linkerd)?

Difficulty: Hard

Service mesh: infrastructure layer for service-to-service communication

Problems solved:

mTLS between services (no app code change)

Observability (metrics, traces, logs) automatically

Traffic management (retries, timeouts, circuit breaking)

Load balancing (L7, intelligent)

Architecture:

Data plane: sidecar proxies (Envoy) alongside each service

Control plane: manage/configure all sidecar proxies

Istio (most popular):

Sidecar: Envoy proxy injected into every pod
Control: istiod (Pilot, Citadel, Galley)

Features:

mTLS: automatic, zero-touch encryption between services
Traffic management: VirtualService, DestinationRule
Canary deploy: 90% to v1, 10% to v2
Circuit breaker: connectionPool, outlierDetection
Observability: Prometheus, Jaeger, Kiali

Linkerd:

Lighter, simpler than Istio
Rust-based sidecar (ultralight, fast)
Less features but easier to operate

Without service mesh:

App libraries (Netflix OSS: Hystrix, Ribbon, Eureka)
Problem: per-language, per-team implementation

When to use:

Many microservices (10+) communicating
Security compliance (zero-trust networking)
When observability without code changes is needed
Added complexity: worth it at scale (20+ services)

Q84. What is DNS load balancing?

Difficulty:
Medium

DNS load balancing: return multiple IPs for a hostname
Client picks one (usually first)
Inherent load distribution across IPs

Round-robin DNS:

api.example.com → [10.0.0.1, 10.0.0.2, 10.0.0.3]
Each query returns IPs in different order (rotating)
Simple, but no health checking
Clients may cache IP and keep hitting dead server

Weighted DNS:

70% → server1, 30% → server2
Used for: traffic shifting, canary deploys, blue-green

GeoDNS:

Return different IPs based on client location
US clients → US servers, EU clients → EU servers
Providers: AWS Route 53, Cloudflare, Akamai

Anycast:

Same IP announced from multiple locations (BGP)
Client routed to nearest announcement
Used by: Cloudflare (1.1.1.1), Google (8.8.8.8), CDNs
Most transparent (no DNS tricks)

DNS TTL impact:

Low TTL (30s): quick failover but more DNS queries
High TTL (300s): fewer queries but slow failover

0 TTL: defeats caching, high DNS load

AWS Route 53:

- Health-check-based failover
- Latency-based routing (nearest region)
- Weighted routing (A/B testing)
- Geolocation routing

Q85. What is TCP socket options?

Difficulty: Hard

```
// TCP socket options important for performance

import (
    "net"
    "syscall"
    "golang.org/x/net/netopt" // or use syscall directly
)

// SO_REUSEADDR: allow bind to same addr after restart
// Set automatically by Go's net.Listen

// TCP_NODELAY: disable Nagle's algorithm (reduce latency)
// Nagle: buffer small packets until ACK received
// TCP_NODELAY: send immediately (good for interactive protocols)
conn, _ := net.Dial("tcp", "server:8080")
tcpConn := conn.(*net.TCPConn)
tcpConn.SetNoDelay(true) // Go method

// SO_KEEPALIVE: enable keepalive probes
tcpConn.SetKeepAlive(true)
tcpConn.SetKeepAlivePeriod(30 * time.Second)

// SO_LINGER: control close behavior
// Default: close returns immediately, OS sends FIN in background
// Linger(true, 0): RST instead of FIN (aggressive close)
// Linger(true, 5): block until all data sent or 5s timeout

// SO_RCVBUF / SO_SNDBUF: socket buffer sizes
// Tune for high-bandwidth connections
// Go default: 4KB, system max: /proc/sys/net/core/rmem_max

// SO_REUSEPORT: multiple processes listen on same port (load balance at kernel)
// Useful for multi-process servers without lock contention
```

Q86. What is QUIC protocol?

Difficulty: Hard

QUIC: Quick UDP Internet Connections (Google → IETF standard)
 Used by: HTTP/3, Google services, Cloudflare
 Transport: UDP (not TCP)

Problems solved:

1. TCP HOL (Head-of-Line) blocking:
 - One lost packet blocks ALL streams on connection
 - QUIC: streams are independent, loss only blocks that stream
2. Connection establishment:

TCP + TLS 1.3: 1 RTT (after TCP 3-way handshake = 1.5 RTT total)
 QUIC: 1 RTT first connection, 0-RTT resumption

3. Connection migration:

TCP: new connection if IP changes (mobile switching WiFi→4G)
 QUIC: connection ID-based, survives IP change

4. Middlebox ossification:

TCP: deep packet inspection by middleboxes, hard to evolve
 QUIC: encrypted (only QUIC header visible), evolvable

QUIC features:

Built-in TLS 1.3 (no separate handshake)
 Stream multiplexing (like HTTP/2 but no HOL blocking)
 Loss recovery (NACK, selective ACK)
 Forward Error Correction (FEC) optional
 Connection IDs (survives IP change)
 Crypto: ChaCha20-Poly1305 or AES-128-GCM

Performance:

~50ms faster page loads (Google data)
 ~15% better on lossy networks (mobile)

Go: quic-go library (cloudflare/quiche wrapper also available)

Q87. What is mTLS (Mutual TLS)?

Difficulty: Hard

mTLS: both client and server authenticate with certificates

Normal TLS:

Server → Client: "Here's my certificate"
 Client verifies server cert
 Server trusts all clients (auth via password, token, etc.)

mTLS:

Server → Client: "Here's my certificate"
 Client → Server: "Here's MY certificate"
 Both verify each other's certificate
 No username/password needed → certificate IS the identity

Use cases:

- Microservices: service-to-service authentication (Istio)
- API clients (banking APIs, partner integrations)
- Zero-trust networking
- IoT device authentication

Implementation in Go:

```
// Server
tlsConfig := &tls.Config{
    ClientAuth: tls.RequireAndVerifyClientCert,
    ClientCAs: loadCAPool("ca.pem"), // CA that signed client certs
}

// Client
cert, _ := tls.LoadX509KeyPair("client.crt", "client.key")
tlsConfig := &tls.Config{
    Certificates: []tls.Certificate{cert},
```

```
    RootCAs: loadCAPool("ca.pem"), // CA that signed server cert
}
```

Certificate management:

Manual: openssl, CFSSL

Automated: cert-manager (Kubernetes), Vault PKI

Service mesh: Istio manages certs automatically (SPIFFE/SPIRE)

Q88. What is HTTP/2 Server Push (deprecated/replaced)?

Difficulty:
Medium

HTTP/2 Server Push: server proactively sends resources before client asks
 Client requests /index.html
 Server pushes /style.css and /app.js alongside HTML response
 Client uses pushed resources when HTML references them

Problems:

Browser may already have resources cached → wasted bandwidth

Hard to implement correctly

Chrome removed push in 2022 (caused more harm than good)

Replace with:

Early Hints (103): server sends Link headers early

HTTP/1.1 103 Early Hints

Link: </style.css>; rel=preload; as=style

Preload links in HTML:

<link rel="preload" href="/style.css" as="style">

These allow browser to decide (cache-aware)

Much more reliable than push

Q89. What is connection pooling?

Difficulty:
Medium

```
// Connection pool: reuse established connections
// Avoid: handshake overhead, TLS overhead, connection limit

// HTTP client pool (default in Go)
transport := &http.Transport{
    MaxIdleConns:      100,           // total idle connections
    MaxIdleConnsPerHost: 10,         // per host
    MaxConnsPerHost:    100,         // total per host
    IdleConnTimeout:    90 * time.Second,
    TLSHandshakeTimeout: 10 * time.Second,
    DisableCompression: false,
}
client := &http.Client{Transport: transport, Timeout: 30 * time.Second}

// Database pool (pgxpool)
pool, _ := pgxpool.New(ctx, dsn+"?pool_max_conns=10&pool_min_conns=2")

// Redis pool (built into go-redis)
rdb := redis.NewClient(&redis.Options{
    PoolSize: 10, MinIdleConns: 3,
})

// Pool sizing:
```

```
// Too small: requests wait for connection (latency)
// Too large: DB/server overwhelmed, resource waste
// Rule of thumb: start with CPU_cores × 2 for CPU-bound services
//               start with 10-20 for I/O-bound services
// Monitor: pool wait time, connection errors
```

Q90. What is Envoy proxy?

Difficulty:
Medium

Envoy: high-performance L7 proxy (C++, cloud-native)
Used by: Istio, AWS App Mesh, Google Cloud Traffic Director

Features:

```
HTTP/1.1, HTTP/2, HTTP/3, gRPC
Service discovery (static, DNS, EDS/xDS API)
Load balancing (round-robin, least-requests, ring-hash, random)
Observability: stats, access logs, distributed tracing (Zipkin, Jaeger)
Circuit breaking: max connections, max requests, outlier detection
Retries with backoff
Rate limiting (global via Rate Limit Service)
Health checking (active + passive)
```

xDS API: dynamic configuration protocol

```
EDS: endpoint discovery (which backends are up)
CDS: cluster discovery (backend groups)
LDS: listener discovery (ports to listen on)
RDS: route discovery (URL routing rules)
```

Envoy as sidecar (service mesh):

```
Intercept all traffic via iptables rules
Apply: mTLS, retries, tracing, metrics
Report to control plane (Istiod for Istio)
```

Admin API:

```
curl localhost:9901/stats/prometheus # metrics
curl localhost:9901/clusters         # backend status
curl localhost:9901/config_dump      # full config
```

Q91. What is Wireshark / packet capture?

Difficulty:
Medium

```
# tcpdump: command line packet capture
tcpdump -i eth0 # capture all traffic on eth0
tcpdump -i eth0 port 443 # HTTPS traffic
tcpdump -i eth0 host 10.0.0.1 # specific host
tcpdump -i eth0 -w capture.pcap # save to file
tcpdump -i eth0 -r capture.pcap # read from file
```

Useful filters

```
tcpdump 'tcp port 80 and host 10.0.0.1'
tcpdump 'tcp[tcpflags] & tcp-syn != 0' # SYN packets only
tcpdump 'udp port 53' # DNS
```

SSL/TLS decode (with private key)

```
tcpdump -i eth0 -w capture.pcap port 443
wireshark capture.pcap # Preferences → RSA keys → add server key
# ss (socket statistics, modern replacement for netstat)
```

```

ss -tlnp          # listening TCP ports with process
ss -s             # socket summary
ss -tp           # established connections with process
ss -tnp | grep :5432 # who's connected to PostgreSQL

# netstat (older)
netstat -tlnp      # listening ports
netstat -an        # all connections

```

Q92. ### Q92–Q150: Remaining Networking Questions

Difficulty:
Medium

Q	Topic
Q92	SSH tunneling for database access
Q93	DHCP lease process
Q94	ARP (Address Resolution Protocol)
Q95	ICMP and ping
Q96	Network namespaces in Linux
Q97	iptables chains and rules
Q98	Linux socket programming
Q99	Kernel bypass networking (DPDK)
Q100	CORS (Cross-Origin Resource Sharing)
Q101	JWT tokens and HTTP authentication
Q102	OAuth 2.0 flows
Q103	Cookie security attributes
Q104	Content Security Policy
Q105	HTTP compression (gzip, brotli, zstd)
Q106	TCP slow start and bandwidth delay product
Q107	Network performance tuning (sysctl)
Q108	Multicast and broadcast
Q109	MPLS networking
Q110	SD-WAN concepts
Q111	Network monitoring: Prometheus + Grafana
Q112	OpenTelemetry distributed tracing
Q113	Log aggregation: ELK / Loki
Q114	API Gateway patterns
Q115	Circuit breaker at network layer
Q116	Bulkhead pattern in networking
Q117	Retry with jitter patterns
Q118	Network chaos engineering
Q119	EBPF for observability

Q120	Kubernetes network policies
Q121	Kubernetes ingress controllers
Q122	Service account and pod networking
Q123	NodePort vs ClusterIP vs LoadBalancer
Q124	CoreDNS in Kubernetes
Q125	IPVS vs iptables in Kubernetes
Q126	Calico vs Flannel vs Cilium
Q127	Kubernetes Gateway API
Q128	Zero-trust networking principles
Q129	Network segmentation patterns
Q130	API rate limiting strategies
Q131	HTTP content negotiation
Q132	Long polling vs WebSocket vs SSE
Q133	GraphQL subscription transport
Q134	gRPC streaming patterns
Q135	Protocol Buffers encoding
Q136	JSON:API spec
Q137	OpenAPI specification
Q138	API versioning strategies
Q139	Backward compatibility in APIs
Q140	API pagination patterns
Q141	ETL vs streaming data ingestion
Q142	Event-driven architecture patterns
Q143	AsyncAPI specification
Q144	Network reliability: SLI, SLO, SLA
Q145	Chaos engineering for network
Q146	Blue-green and canary via DNS
Q147	Traffic mirroring (shadowing)
Q148	Sidecar vs per-node DaemonSet proxy
Q149	Multi-region networking patterns
Q150	SDE2 networking interview checklist

Extended Questions (Q93–Q150)

Q93. What is HTTP/2 and how does it differ from HTTP/1.1?

Difficulty:
Medium

HTTP/1.1 problems:

- Head-of-line blocking: requests queue sequentially per connection
- Multiple connections needed for parallelism (browsers open 6-8)
- Header redundancy: same headers sent with every request
- No server push

HTTP/2 solutions:

- Multiplexing: multiple streams on ONE connection (no HOL blocking)
- Header compression: HPACK algorithm (70-90% size reduction)
- Binary framing: efficient parsing (vs text in HTTP/1.1)
- Stream prioritization: important resources first
- Server push: proactively send resources before client asks

HTTP/2 concepts:

Frame: smallest unit of communication (headers, data, settings)

Stream: bidirectional sequence of frames (virtual connection)

Connection: one TCP connection carries multiple streams

Stream ID: client odd (1,3,5...), server even (2,4,6...)

Performance:

HTTP/1.1: 6 parallel connections × 1 request = 6 concurrent

HTTP/2: 1 connection × N streams = N concurrent (no limit)

Latency: 30-50% faster than HTTP/1.1

Header: 85-95% smaller (HPACK)

Q94. What is HTTP/3 and QUIC?**Difficulty: Hard**

HTTP/3: HTTP over QUIC (not TCP)

QUIC (Quick UDP Internet Connections):

- Built on UDP (not TCP)
- TLS 1.3 built-in (no separate TLS handshake)
- Connection migration: works across IP changes (mobile roaming)
- Independent streams: no head-of-line blocking at transport layer
- 0-RTT connection: reconnect without handshake (known servers)

HTTP/2 vs HTTP/3:

HTTP/2 problem: multiplexing over TCP → TCP HOL blocking

One lost packet → ALL streams wait for retransmit!

HTTP/3 solution: QUIC manages per-stream retransmit

One lost packet → only that stream waits

QUIC handshake:

First connection: 1-RTT (vs 2-RTT for TCP+TLS)

Known server: 0-RTT (send data immediately)

Performance:

Poor network (packet loss): HTTP/3 >> HTTP/2

Good network: HTTP/3 ≈ HTTP/2

Mobile networks: HTTP/3 much better (IP changes handled)

Support: Chrome, Firefox, Safari, nginx, Cloudflare, Google

Q95. What is WebSocket protocol?**Difficulty:**
Medium

WebSocket: full-duplex communication over single TCP connection
 Upgrade from HTTP → persistent bidirectional channel

Handshake:

Client sends HTTP Upgrade request:

```
GET /ws HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhliHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Server responds:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTx9kYGzzhZRbK+x0o=
```

Frame format:

```
1 bit FIN, 3 bits RSV, 4 bits opcode, 1 bit MASK,
7 bits payload length, (optional extended length), masking key, payload
```

Opcodes:

```
0x1: text frame (UTF-8)
0x2: binary frame
0x8: close
0x9: ping
0xA: pong
```

Use cases: chat, live dashboards, gaming, collaborative editing

vs SSE: WebSocket = bidirectional; SSE = server→client only

vs HTTP polling: WebSocket = push; polling = pull (inefficient)

Q96. What is gRPC and Protocol Buffers?**Difficulty: Hard**

// Protocol Buffers: binary serialization format

```
syntax = "proto3";
```

```
message User {
  int64 id = 1;
  string name = 2;
  string email = 3;
  repeated string roles = 4;
}
```

```
service UserService {
  rpc GetUser(GetUserRequest) returns (User);
  rpc ListUsers(ListUsersRequest) returns (stream User); // server streaming
  rpc BulkCreate(stream CreateUserRequest) returns (CreateResponse); // client streaming
  rpc Chat(stream Message) returns (stream Message); // bidirectional streaming
}
```

gRPC advantages over REST:

Performance: protobuf binary (5-10x smaller than JSON)

Type safety: generated code, compile-time checks
Streaming: native bidirectional streaming
Code generation: client/server stubs auto-generated

gRPC over HTTP/2:

Single TCP connection, multiple streams
Each RPC = one HTTP/2 stream
Header compression via HPACK
Multiplexed (no HOL blocking per call)

gRPC status codes:

OK(0), CANCELLED(1), UNKNOWN(2), INVALID_ARGUMENT(3),
NOT_FOUND(5), ALREADY_EXISTS(6), PERMISSION_DENIED(7),
RESOURCE_EXHAUSTED(8), FAILED_PRECONDITION(9),
UNAVAILABLE(14), DEADLINE_EXCEEDED(4)

When to use gRPC:

Internal microservice communication
Mobile clients (bandwidth matters)
Streaming (real-time data)
Multiple language services

When to use REST:

Public APIs (browser/curl friendly)
Simple CRUD
Caching at CDN/proxy level

Q97. What is mTLS (mutual TLS)?

Difficulty: Hard

TLS: server proves identity to client (one-way)
mTLS: BOTH server AND client prove identity (two-way)

Handshake (mTLS):

1. Client hello (supported ciphers)
2. Server hello + server certificate
3. Server requests client certificate
4. Client sends client certificate
5. Both verify certificates against CA
6. Session keys established
7. Encrypted communication begins

Certificate chain:

Root CA → Intermediate CA → Leaf certificate
Trust: client trusts Root CA → verifies server/client certs

Use cases:

Service mesh (Istio): mTLS between all microservices
API authentication: client cert instead of API key
Internal services: no external auth needed
Zero trust networking: every service authenticated

In Go:

```
tls.Config{
    ClientAuth: tls.RequireAndVerifyClientCert,
    ClientCAs: certPool, // CA pool for client cert verification
    Certificates: []tls.Certificate{serverCert},
}
```

Operational complexity:

Certificate rotation: automate (short-lived certs, SPIFFE/SPIRE)

Certificate distribution: Vault PKI, cert-manager

Q98. What is CORS (Cross-Origin Resource Sharing)?

Difficulty:
Medium

CORS: browser security mechanism controlling cross-origin requests

Origin = scheme + host + port

Same-origin: `https://app.com` → `https://app.com/api` (OK)

Cross-origin: `https://app.com` → `https://api.other.com` (blocked by browser)

CORS headers:

Response headers (server sets):

Access-Control-Allow-Origin: `https://app.com` (or `*`)

Access-Control-Allow-Methods: GET, POST, PUT, DELETE

Access-Control-Allow-Headers: Content-Type, Authorization

Access-Control-Allow-Credentials: true (for cookies/auth)

Access-Control-Max-Age: 86400 (preflight cache: 24h)

Preflight request (OPTIONS):

Browser sends OPTIONS before actual request if:

- Non-simple method (not GET/POST/HEAD)
- Custom headers (e.g., Authorization)
- Content-Type not simple (application/json triggers preflight)

CORS in Go:

```
func corsMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Access-Control-Allow-Origin", "https://myapp.com")
        w.Header().Set("Access-Control-Allow-Methods", "GET,POST,PUT,DELETE")
        w.Header().Set("Access-Control-Allow-Headers", "Content-Type,Authorization")
        if r.Method == http.MethodOptions {
            w.WriteHeader(http.StatusNoContent); return
        }
        next.ServeHTTP(w, r)
    })
}
// Or use: github.com/rs/cors
```

Q99. What is DNS resolution process?

Difficulty:
Medium

DNS resolution steps for example.com:

1. Browser cache: check local browser DNS cache (TTL-based)
2. OS cache: `/etc/hosts`, OS DNS cache
3. Recursive resolver (ISP/8.8.8.8):
 - Check resolver cache
 - If miss: query root nameservers
4. Root nameservers (.):
 - Returns NS records for .com TLD
5. TLD nameservers (.com):
 - Returns NS records for example.com
6. Authoritative nameserver (example.com):
 - Returns A/AAAA record for example.com

Caching:

Each response has TTL (seconds)
Low TTL (30s): fast failover, more DNS queries
High TTL (86400s): cached 24h, slow failover

Record types:

A: IPv4 address (93.184.216.34)
AAAA: IPv6 address
CNAME: alias to another name (www → example.com)
MX: mail server
NS: nameserver
TXT: arbitrary text (SPF, DKIM, verification)
SRV: service location (host, port, priority, weight)
PTR: reverse DNS (IP → hostname)

DNS over HTTPS (DoH): encrypt DNS queries (privacy)

DNSSEC: sign DNS records (authenticity, prevent spoofing)

Q100. What is load balancing algorithms?**Difficulty:**
Medium**Round Robin:**

Requests distributed sequentially: R1→S1, R2→S2, R3→S3, R4→S1...
Simple, even distribution
Problem: ignores server capacity and current load

Weighted Round Robin:

Servers have weights: S1(3), S2(2), S3(1)
S1 gets 50%, S2 gets 33%, S3 gets 17%
Use: different server capacities

Least Connections:

Route to server with fewest active connections
Better for variable request duration
Use: long-lived connections (WebSocket, streaming)

Least Response Time:

Route to server with lowest average response time
Adaptive to real-time performance
Overhead: must track response times

IP Hash (Sticky Sessions):

hash(client_IP) % servers → always same server
Use: stateful apps, session consistency
Problem: uneven if many users from same IP (NAT)

Random:

Random server selection
Simple, surprisingly effective
"Power of Two Choices": pick 2 random, choose less loaded

Consistent Hashing:

Used in distributed caches (Redis Cluster, Cassandra)
Minimizes redistribution when servers added/removed
Virtual nodes for even distribution

Q101. What is TCP keep-alive?**Difficulty:**
Medium

TCP keep-alive: detect dead connections (NAT timeout, crashed hosts)

Problem: idle TCP connection looks alive to both ends
 But NAT table entry expired → packets silently dropped
 Server holds resources for phantom connections

TCP keep-alive mechanism:

After idle_time (default 2h): send keep-alive probe (ACK)
 If no response: retry every interval (75s) for count times (9)
 If still no response: close connection (ETIMEDOUT/ECONNRESET)

Linux defaults:

net.ipv4.tcp_keepalive_time = 7200 (2h before first probe)
 net.ipv4.tcp_keepalive_intvl = 75 (75s between probes)
 net.ipv4.tcp_keepalive_probes = 9 (9 probes before giving up)

Production tuning (for microservices):

tcp_keepalive_time = 60 (1 min)
 tcp_keepalive_intvl = 10 (10s)
 tcp_keepalive_probes = 3 (3 probes = 30s to detect dead)

In Go:

```
conn, _ := net.DialTimeout("tcp", addr, timeout)
tcpConn := conn.(*net.TCPConn)
tcpConn.SetKeepAlive(true)
tcpConn.SetKeepAlivePeriod(30 * time.Second)
```

HTTP client in Go:

```
transport := &http.Transport{
    DialContext: (&net.Dialer{
        KeepAlive: 30 * time.Second,
    }).DialContext,
}
```

Q102. What are HTTP status codes every developer must know?**Difficulty: Easy**

2xx Success:

200 OK: standard success
 201 Created: resource created (POST/PUT), include Location header
 204 No Content: success, no body (DELETE, PUT update)
 206 Partial Content: range request (video streaming)

3xx Redirection:

301 Moved Permanently: cached forever, change bookmarks
 302 Found: temporary redirect, don't cache
 304 Not Modified: cached version still valid (ETag/Last-Modified)
 307 Temporary Redirect: preserve method (POST stays POST)
 308 Permanent Redirect: preserve method, permanent

4xx Client Error:

400 Bad Request: invalid request syntax/parameters
 401 Unauthorized: authentication required (not authenticated)
 403 Forbidden: authenticated but not authorized
 404 Not Found: resource doesn't exist

405 Method Not Allowed: GET on POST-only endpoint
409 Conflict: state conflict (duplicate, version mismatch)
410 Gone: permanently deleted (vs 404)
422 Unprocessable Entity: semantic validation failure
429 Too Many Requests: rate limited (include Retry-After)

5xx Server Error:

500 Internal Server Error: generic server error
502 Bad Gateway: upstream returned invalid response
503 Service Unavailable: server overloaded/maintenance
504 Gateway Timeout: upstream didn't respond in time

Q103. What is NAT (Network Address Translation)?

Difficulty:
Medium

NAT: translates private IPs to public IPs

Problem: IPv4 exhaustion (4.3B addresses, 8B+ devices)

Solution: private IP ranges + NAT gateway

Private IP ranges (RFC 1918):

10.0.0.0/8	(10.x.x.x) - 16M addresses
172.16.0.0/12	(172.16-31.x.x) - 1M addresses
192.168.0.0/16	(192.168.x.x) - 65K addresses

NAT types:

SNAT (Source NAT): outbound traffic, change source IP
DNAT (Destination NAT): inbound traffic, change destination IP
PAT (Port Address Translation) / NAPT: many-to-one (most common)

How PAT works:

192.168.1.5:54321 → google.com:443
NAT gateway rewrites to: 1.2.3.4:10001 → google.com:443
NAT table: 10001 ↔ 192.168.1.5:54321
Return packet: 1.2.3.4:10001 → translated back

NAT problems:

No inbound connections (port forwarding needed)
Breaks IPsec (changes IP)
Stateful: must track connections
Breaks some protocols (FTP passive mode, SIP)
TCP keep-alive: NAT entries expire silently

Q104. What is BGP (Border Gateway Protocol)?

Difficulty: Hard

BGP: the routing protocol of the internet

Connects autonomous systems (AS): ISPs, cloud providers, enterprises
Each AS has an AS number (ASN): e.g., Google=15169, AWS=16509

How BGP works:

BGP peers exchange routing tables (AS paths)
Each router advertises: "I can reach X.X.X.X/Y via AS1→AS2→AS3"
Routers choose best path (shortest AS path, various attributes)

BGP attributes:

AS_PATH: list of ASes the route traversed (loop prevention)
NEXT_HOP: next router to send packets to

LOCAL_PREF: prefer within AS (higher = preferred)
 MED: Multi-Exit Discriminator (hint to neighbors)
 COMMUNITY: tags for policy application

BGP for cloud (AWS/GCP):

VPN: BGP over IPsec (dynamic routing, learn routes automatically)
 Direct Connect: BGP session between on-prem and AWS

BGP security issues:

BGP hijacking: AS advertises someone else's prefix (attack/misconfiguration)
 2010: Pakistan Telecom → YouTube blackhole
 BGPsec: cryptographic path validation (not widely deployed)
 RPKI: Resource Public Key Infrastructure (route origin validation)

Q105. What is network namespaces in Linux?

Difficulty: Hard

```
# Network namespace: isolated network stack
# Used by: Docker containers, Kubernetes pods, VPNs

# Create namespace
ip netns add myns

# Run command in namespace
ip netns exec myns ip addr
ip netns exec myns bash # shell in namespace

# Container has own:
# - Network interfaces (lo, eth0)
# - Routing table
# - iptables rules
# - /proc/net/* entries

# Connect two namespaces via veth pair
ip link add veth0 type veth peer name veth1
ip link set veth1 netns myns
ip addr add 192.168.100.1/24 dev veth0
ip netns exec myns ip addr add 192.168.100.2/24 dev veth1
ip link set veth0 up
ip netns exec myns ip link set veth1 up

# Now can ping between namespaces:
ping 192.168.100.2 # host → container
ip netns exec myns ping 192.168.100.1 # container → host

# Docker uses this: each container = network namespace
# Kubernetes: pod = shared network namespace (all containers in pod share IP)
```

Q106. What is iptables and packet filtering?

Difficulty: Hard

```
# iptables: Linux kernel packet filtering/NAT framework
# Tables: filter, nat, mangle, raw
# Chains: INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING

# List rules
iptables -L -n -v # filter table
iptables -t nat -L -n -v # NAT table
```

```

iptables -t nat -L -n -v --line-numbers # with line numbers

# Allow/drop traffic
iptables -A INPUT -p tcp --dport 22 -j ACCEPT # allow SSH
iptables -A INPUT -p tcp --dport 8080 -j ACCEPT # allow port 8080
iptables -A INPUT -j DROP # drop everything else

# Docker uses iptables extensively:
# DOCKER chain: container port mappings
# DOCKER-USER chain: custom rules (insert here for Docker traffic)
# MASQUERADE: SNAT for outbound container traffic

# Block Docker container from reaching host
iptables -I DOCKER-USER -i docker0 -j DROP
iptables -I DOCKER-USER -i docker0 -d 192.168.1.100 -j DROP

# View Docker-related rules
iptables -t nat -L DOCKER -n -v
iptables -L DOCKER-USER -n -v

# nftables: modern replacement for iptables
# Kubernetes: kube-proxy uses iptables (or IPVS) for service routing

```

Q107. What is connection pooling in HTTP clients?

Difficulty:
Medium

```

// Go HTTP client reuses connections by default
// But must configure properly

transport := &http.Transport{
    // Max connections per host (default 100)
    MaxIdleConnsPerHost: 100,
    MaxIdleConns:        1000,
    MaxConnsPerHost:     200,

    // Idle timeout (close connection if idle this long)
    IdleConnTimeout:     90 * time.Second,

    // Timeouts
    TLSHandshakeTimeout: 10 * time.Second,
    ResponseHeaderTimeout: 10 * time.Second,
    ExpectContinueTimeout: 1 * time.Second,

    DialContext: (&net.Dialer{
        Timeout: 30 * time.Second,
        KeepAlive: 30 * time.Second,
    }).DialContext,

    ForceAttemptHTTP2: true, // enable HTTP/2
}

client := &http.Client{
    Transport: transport,
    Timeout: 30 * time.Second, // total request timeout
}

// Important: close response body or connections leak

```

```
resp, err := client.Get(url)
if err != nil { return err }
defer resp.Body.Close()
io.Copy(io.Discard, resp.Body) // drain body (important for keep-alive)

// NEVER create new http.Client per request!
// Share one client (or transport) across goroutines
```

Q108. What is Rate Limiting algorithms?

Difficulty: Hard

Fixed Window:

- Count requests in fixed time window (e.g., 100/minute)
- Simple, predictable
- Problem: boundary burst – 100 at 0:59, 100 at 1:01 = 200 in 2 seconds

Sliding Window (Log):

- Track timestamp of each request, count in last N seconds
- Accurate, no boundary burst
- Memory: stores one timestamp per request (expensive at scale)

Sliding Window (Counter):

- Weighted average of current and previous window counts
- $\text{current_window_count} + \text{prev_window_count} \times (1 - \text{elapsed}/\text{window_size})$
- Approximate but memory efficient

Token Bucket:

- Tokens accumulate at rate R (max burst B tokens)
- Request consumes 1 token; rejected if empty
- Allows bursting up to B tokens
- Use: API rate limits that allow occasional bursts

Leaky Bucket:

- Requests enter bucket (queue); leak at constant rate
- Smooths out bursts (output is steady rate)
- Use: traffic shaping (QoS)

Comparison:

- Token bucket: good for bursty traffic
- Leaky bucket: smooth output
- Sliding window: most accurate rate limiting
- Fixed window: simplest, cache-friendly

Implementation: Redis (INCR + EXPIRE), or golang.org/x/time/rate

Q109. What is CDN (Content Delivery Network)?

Difficulty: Medium

CDN: geographically distributed servers serving content from edge
User → nearest CDN PoP → cached response (or origin if miss)

How CDN caching works:

- First request to PoP → cache miss → fetch from origin
- Subsequent requests → cache hit (sub-millisecond)
- Cache-Control headers control CDN behavior:
 - Cache-Control: max-age=3600 (cache 1 hour)
 - Cache-Control: public (CDN can cache)
 - Cache-Control: no-store (never cache)

Cache invalidation:

TTL expiry (passive)
 API purge (Cloudflare, CloudFront APIs)
 Cache tags / surrogate keys (purge by tag)
 Versioned URLs: /static/app.v1.2.js (immutable, max TTL)

CDN features:

Static assets: images, CSS, JS, fonts
 Dynamic acceleration: optimize routing to origin
 WAF: web application firewall at edge
 DDoS protection: absorb at edge
 SSL termination: certificate at edge
 HTTP/3, QUIC: enabled at edge even if origin HTTP/1.1

Major CDNs: Cloudflare, CloudFront (AWS), Fastly, Akamai

CDN caching layers:

Browser cache → CDN edge → CDN origin shield → Origin server

Q110. What is network troubleshooting commands?

Difficulty:
Medium

```
# Connectivity
ping google.com                # ICMP reachability
ping -c 4 -i 0.5 8.8.8.8       # 4 pings, 0.5s interval
traceroute google.com          # hop-by-hop path
mtr google.com                 # continuous traceroute

# DNS
nslookup example.com           # DNS lookup
dig example.com                 # detailed DNS
dig +short example.com          # just IPs
dig @8.8.8.8 example.com        # use specific DNS server
dig -x 1.2.3.4                 # reverse DNS
host example.com               # quick lookup

# Ports/connections
ss -tlnp                      # listening TCP ports + process
ss -tnp                        # established connections
netstat -tlnp                 # (older alternative to ss)
nmap -p 80,443,8080 example.com # port scan
curl -v https://example.com     # HTTP with headers
telnet example.com 80           # raw TCP test

# Traffic capture
tcpdump -i eth0 port 80        # capture HTTP traffic
tcpdump -i eth0 host 1.2.3.4    # traffic to/from IP
tcpdump -i eth0 -w capture.pcap # save to file
wireshark capture.pcap         # analyze in Wireshark

# Interface
ip addr                       # interfaces and IPs
ip route                      # routing table
ip link set eth0 up/down      # enable/disable interface

# Bandwidth
iperf3 -s                     # server mode
```

```
iperf3 -c server-ip # test bandwidth
```

Q111. What is SSL/TLS certificate chain?

Difficulty:
Medium

Certificate chain (chain of trust):

Root CA → Intermediate CA → Leaf (server) certificate

Root CA:

Self-signed (trusted by OS/browser)
Never directly signs server certs (security isolation)
Example: DigiCert Root CA, Let's Encrypt ISRG Root X1

Intermediate CA:

Signed by Root CA
Signs leaf certificates
Offline Root CA → minimizes Root CA exposure

Leaf (server) certificate:

Contains: domain name, public key, validity, issuer
Signed by Intermediate CA
Presented by server during TLS handshake

Verification process:

1. Server sends: leaf cert + intermediate cert
2. Client: trusts intermediate? → check against Root CA
3. Client: verify signature chain up to trusted Root
4. Client: check expiry, hostname, revocation (OCSP/CRL)

Certificate transparency:

All issued certs logged to public CT logs
Monitors can detect mis-issuance

OCSP stapling:

Server pre-fetches revocation status from CA
Includes in TLS handshake (stapled response)
Faster + privacy (client doesn't contact CA)

In Go:

`tls.Config{}` → auto-verifies chain using system root CAs
`x509.CertPool{}` for custom CAs

Q112. What is HTTP caching headers?

Difficulty:
Medium

Cache-Control (response):

<code>max-age=3600</code>	cache for 3600 seconds (relative)
<code>s-maxage=3600</code>	CDN cache duration (overrides max-age for CDNs)
<code>no-cache</code>	revalidate before using cached copy
<code>no-store</code>	never cache (sensitive data)
<code>public</code>	any cache can store
<code>private</code>	only browser cache (not CDN)
<code>immutable</code>	content will never change (hash in URL)
<code>stale-while-revalidate=60</code>	serve stale while fetching fresh

ETag: "version-hash"

Opaque identifier for resource version

Client sends: If-None-Match: "version-hash"
Server: 304 Not Modified if unchanged, 200 + body if changed

Last-Modified: Thu, 01 Jan 2024 00:00:00 GMT
Client sends: If-Modified-Since: Thu, 01 Jan 2024 00:00:00 GMT
Server: 304 if not modified since

Vary: Accept-Encoding
CDN caches separate copies per Accept-Encoding value
Vary: Accept-Language → separate cache per language

Ideal strategy:
HTML: Cache-Control: no-cache (always revalidate)
CSS/JS (versioned): Cache-Control: public, max-age=31536000, immutable
API JSON: Cache-Control: private, max-age=60
User data: Cache-Control: no-store

Q113. ### Q113–Q150: Additional Networking Questions

Difficulty:
Medium

Q114. What is HTTPS and TLS handshake (TLS 1.3)?

Difficulty:
Medium

TLS 1.3 handshake (1-RTT, down from 2-RTT in 1.2):
1. Client Hello: supported ciphers, key_share (DH public key), SNI
2. Server Hello: chosen cipher, key_share, certificate, Finished
3. Client: Finished (handshake complete)
→ Data flows after 1 round trip!

TLS 1.3 vs 1.2:
Removed: RSA key exchange, weak ciphers (RC4, DES, 3DES)
Required: forward secrecy (ephemeral DH keys)
0-RTT: resumption without handshake (replay attack risk)
Faster: 1-RTT vs 2-RTT

Cipher suite (TLS 1.3):
TLS_AES_128_GCM_SHA256
TLS_AES_256_GCM_SHA384
TLS_CHACHA20_POLY1305_SHA256

ALPN: Application-Layer Protocol Negotiation
Client advertises: h2, http/1.1
Server picks: h2 (HTTP/2) if supported
Avoids extra round trip for protocol negotiation

Q115. What is service discovery in microservices?

Difficulty:
Medium

Problem: containers/services have dynamic IPs (restarts change IP)

Client-side discovery:
Client queries registry, picks instance, calls directly
Libraries: Eureka (Netflix), Consul client
Pros: client controls load balancing
Cons: client complexity per language

Server-side discovery:

Client calls load balancer, LB queries registry
 Examples: AWS ALB, Kubernetes Service, nginx
 Pros: simple client (just one endpoint)
 Cons: extra hop, LB is single point of failure

DNS-based discovery:

Service registered as DNS SRV records
 Client resolves DNS → gets IP list
 Kubernetes: service.namespace.svc.cluster.local
 Simple, language-agnostic

Health-based removal:

Failing instances removed from registry
 Consul: health checks (TCP, HTTP, script)
 Kubernetes: readiness probe controls Endpoints

Popular tools:

Consul: service mesh + discovery + KV store
 Kubernetes: built-in (Service + kube-dns/CoreDNS)
 Eureka: Netflix, Java-centric
 etcd: distributed KV, used by Kubernetes

Q116. What is network policies in Kubernetes?

Difficulty:
Medium

NetworkPolicy: restrict pod-to-pod traffic
 # Default: all traffic allowed between pods

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: postgres      # applies to postgres pods
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: api    # only api pods can connect
      ports:
        - protocol: TCP
          port: 5432
  egress:
    - to: []             # no outbound (postgres doesn't need to call others)
```

Q117. What is reverse proxy vs forward proxy?

Difficulty:
Medium

Forward proxy: client-side proxy
 Client → Forward Proxy → Internet
 Client knows about proxy, configures it

Use: anonymity, content filtering, bypass geo-restrictions
Examples: Squid, corporate proxy

Reverse proxy: server-side proxy

Client → Reverse Proxy → Backend servers

Client doesn't know about backend servers (sees only proxy)

Use: load balancing, SSL termination, caching, WAF

Examples: nginx, HAProxy, Cloudflare, AWS ALB

nginx as reverse proxy:

```
server {  
    listen 80;  
    server_name api.example.com;  
    location / {  
        proxy_pass http://backend:8080;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header Host $host;  
    }  
}
```

API Gateway: specialized reverse proxy

Features: auth, rate limiting, routing, transformation, analytics

Examples: Kong, AWS API Gateway, Nginx (with plugins)

Q118. What is TCP vs UDP comparison?

Difficulty:
Medium

TCP (Transmission Control Protocol):

Connection-oriented (3-way handshake)

Reliable delivery (ACK + retransmit)

Ordered delivery (sequence numbers)

Flow control (receiver buffer)

Congestion control (slow start, AIMD)

Use: HTTP, HTTPS, SSH, FTP, databases

UDP (User Datagram Protocol):

Connectionless (no handshake)

No delivery guarantee (fire-and-forget)

No ordering

No flow/congestion control

Lightweight header (8 bytes vs TCP 20 bytes)

Use: DNS, video streaming, gaming, VoIP, QUIC, NTP

When UDP is better:

Latency-sensitive (gaming, video calls)

Occasional loss acceptable (stream next frame)

High-frequency small messages (DNS queries)

Broadcast/multicast

When you implement custom reliability (QUIC)

TCP handshake:

SYN → SYN-ACK → ACK (3 messages = 1.5 RTT to establish)

Connection teardown: FIN → FIN-ACK → FIN → ACK

TIME_WAIT: ~2 MSL (Maximum Segment Lifetime, 60-120s) after close

Purpose: ensure stale packets don't confuse new connections

Issue: port exhaustion under high connection rate

Fix: SO_REUSEADDR, tcp_tw_reuse

Q119. What is IP addressing and CIDR?

Difficulty:
Medium

IPv4: 32-bit address (4 octets)

192.168.1.100 → 11000000.10101000.00000001.01100100

CIDR (Classless Inter-Domain Routing):

192.168.1.0/24 → /24 = 24 bits network, 8 bits host

→ 256 addresses (192.168.1.0 - 192.168.1.255)

→ 254 usable (.0=network, .255=broadcast)

Subnet mask:

/24 = 255.255.255.0

/16 = 255.255.0.0

/8 = 255.0.0.0

/32 = 255.255.255.255 (single host)

/0 = 0.0.0.0 (entire internet)

Common subnets:

/24: 254 hosts (small office/service)

/22: 1022 hosts (medium network)

/16: 65534 hosts (VPC/data center)

IPv6: 128-bit address

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Shortened: 2001:db8:85a3::8a2e:370:7334

/64: standard subnet size (2^{64} hosts per subnet)

Loopback: ::1 (vs 127.0.0.1 IPv4)

Link-local: fe80::/10

VPC (AWS): 10.0.0.0/16 (65536 IPs)

Subnet: 10.0.1.0/24 (public), 10.0.2.0/24 (private)

Q120. What is HTTP keep-alive and connection reuse?

Difficulty:
Medium

HTTP/1.0: new TCP connection per request (default)

HTTP/1.1: persistent connections (keep-alive by default)

HTTP/2: multiplexing (many requests per connection)

HTTP/1.1 Keep-Alive:

Connection: keep-alive (client request)

Keep-Alive: timeout=5, max=100 (server response)

Connection: close (when done with connection)

Benefits:

No 3-way handshake for each request

No TLS handshake for each request

Better throughput (pipelining)

Server tuning (nginx):

keepalive_timeout 65; # close idle after 65s

keepalive_requests 1000; # max requests per connection

keepalive_disable none; # enable for all browsers

Client (Go http.Transport):

```
IdleConnTimeout: 90s           # close idle connection
MaxIdleConnsPerHost: 100       # pool size per host
DisableKeepAlives: false      # must be false for pooling
```

Pitfall:

```
Not draining response body → connection not returned to pool!
Always: io.Copy(io.Discard, resp.Body); resp.Body.Close()
```

Q121. What is VPN (Virtual Private Network)?

Difficulty:
Medium

VPN: encrypted tunnel over public internet

Types:

Site-to-Site VPN:

```
Connect two networks (office ↔ data center)
IPsec over internet
AWS VPN Gateway, Cisco ASA
```

Client VPN (Remote Access):

```
User device → VPN server → corporate network
OpenVPN, WireGuard, Cisco AnyConnect, AWS Client VPN
```

Split tunneling:

```
Only corporate traffic goes through VPN
Internet traffic goes direct
Reduces VPN bandwidth
```

Protocols:

```
IPsec: industry standard, complex, hardware support
IKEv2: modern, fast reconnect (mobile-friendly)
OpenVPN: TLS-based, port 443 (firewall-friendly)
WireGuard: modern, fast, simple (~4000 lines of code)
Uses: Curve25519, ChaCha20, Poly1305, BLAKE2s
```

WireGuard vs OpenVPN:

```
WireGuard: faster (kernel module), simpler config, 3-4x throughput
OpenVPN: mature, more features, better firewall traversal
```

AWS Direct Connect vs VPN:

```
VPN: encrypted, over internet, 1-2 Gbps max, latency variable
Direct Connect: dedicated fiber, 1-100 Gbps, predictable latency
```

Q122. ### Q121-Q150: Final Networking Questions

Difficulty:
Medium

Q	Topic
Q121	Anycast routing (Cloudflare, DNS)
Q122	Network latency sources (propagation, transmission, queueing)
Q123	East-west vs north-south traffic in data centers
Q124	Kubernetes CNI plugins (Calico, Flannel, Cilium)
Q125	TCP slow start and congestion control

Q126	WebRTC (STUN, TURN, ICE)
Q127	OAuth 2.0 / OIDC token flow
Q128	JWT (JSON Web Token) structure and validation
Q129	API gateway patterns (rate limiting, auth, routing)
Q130	Zero Trust networking model
Q131	eBPF for network observability
Q132	IPVS vs iptables for Kubernetes services
Q133	Network throughput vs latency optimization
Q134	DNS TTL and caching strategy
Q135	HTTP/2 vs HTTP/1.1 performance comparison
Q136	Certificate rotation (automated, zero downtime)
Q137	Network partition in distributed systems (CAP)
Q138	Socket options (SO_REUSEADDR, SO_REUSEPORT, TCP_NODELAY)
Q139	Health check endpoint design
Q140	Service mesh sidecar pattern (Envoy proxy)
Q141	HTTPS everywhere (HTTP → HTTPS redirect)
Q142	Network observability (metrics, traces, logs)
Q143	Egress traffic control and security
Q144	IP failover and anycast for HA
Q145	DDoS mitigation strategies
Q146	Bandwidth estimation and capacity planning
Q147	Network debugging in Kubernetes
Q148	IPv6 adoption and dual-stack deployment
Q149	Istio traffic management (VirtualService, DestinationRule)
Q150	Production networking checklist

Q123. What is TCP TIME_WAIT and how to handle it?

Difficulty:
Medium

```
# TIME_WAIT: state after connection close, lasts 2*MSL (60-120s)
# Purpose: ensure stale packets don't confuse new connections
# Problem: high-churn servers exhaust ephemeral ports
```

```
# Check TIME_WAIT count
ss -s | grep TIME-WAIT
# or: ss -tn state time-wait | wc -l
```

```
# Mitigation options:
# 1. SO_REUSEADDR (default in most servers): allow port reuse
# 2. tcp_tw_reuse (Linux): reuse TIME_WAIT socket for outbound connections
sysctl -w net.ipv4.tcp_tw_reuse=1
```

```
# 3. Increase ephemeral port range
sysctl -w net.ipv4.ip_local_port_range="1024 65535"

# 4. Reduce TIME_WAIT timeout (risky!)
sysctl -w net.ipv4.tcp_fin_timeout=15 # default 60

# 5. Enable TCP keep-alive to detect dead connections earlier

# High-performance HTTP servers:
# Use persistent connections (HTTP keep-alive)
# Fewer connection teardowns = fewer TIME_WAIT states
# Go HTTP server: automatically uses keep-alive

# In Go HTTP client: reuse connections via transport
transport := &http.Transport{MaxIdleConnsPerHost: 100}
```

Q124. What is SNI (Server Name Indication)?

Difficulty:
Medium

SNI: TLS extension allowing multiple SSL certs on one IP

Problem: IPv4 exhaustion → many virtual hosts share one IP

HTTPS: TLS handshake before HTTP Host header → server doesn't know which cert!

SNI solution:

Client sends hostname in TLS ClientHello (before encryption)

Server selects correct certificate for that hostname

Then TLS handshake proceeds with correct cert

Without SNI:

IP 1.2.3.4 → only one HTTPS virtual host possible

With SNI:

IP 1.2.3.4 → example.com cert (SNI: example.com)

IP 1.2.3.4 → other.com cert (SNI: other.com)

Same IP, different certs!

Privacy concern:

SNI is plaintext in ClientHello → ISP can see which site you visit

ECH (Encrypted Client Hello): encrypts SNI (TLS 1.3 extension)

Supported by: Cloudflare, Firefox

In Go:

```
tls.Config{ServerName: "example.com"} // client sets SNI
tls.Config{GetCertificate: func(info *tls.ClientHelloInfo) (*tls.Certificate, error) {
    return certForHost(info.ServerName) // server selects cert by SNI
}}
```

Q125. What is Kubernetes DNS resolution?

Difficulty:
Medium

```
# Kubernetes DNS: CoreDNS (replaced kube-dns in 1.13)
# Every pod has /etc/resolv.conf:
# nameserver 10.96.0.10 (ClusterIP of kube-dns service)
# search default.svc.cluster.local svc.cluster.local cluster.local
# ndots:5

# Service DNS names:
```

```
# <service>                                (within same namespace)
# <service>.<namespace>                     (cross-namespace)
# <service>.<namespace>.svc                 (FQDN short)
# <service>.<namespace>.svc.cluster.local  (FQDN)

# Examples:
# my-service                               → resolves in current namespace
# my-service.other-ns                      → resolves cross-namespace
# postgres.database.svc.cluster.local     → fully qualified

# Pod DNS:
# <pod-ip-with-dashes>.<namespace>.pod.cluster.local
# 10-0-0-5.default.pod.cluster.local

# Headless service (ClusterIP: None):
# Returns individual Pod IPs instead of Service ClusterIP
# Used for: StatefulSets, direct pod addressing, client-side LB

# DNS troubleshooting in pod:
kubectl exec -it mypod -- nslookup kubernetes.default
kubectl exec -it mypod -- cat /etc/resolv.conf
kubectl exec -it mypod -- curl my-service.my-namespace.svc.cluster.local
```

Q126. What is connection draining (graceful shutdown)?

Difficulty:
Medium

```
// Connection draining: finish in-flight requests before shutdown
// Triggered by: SIGTERM (Kubernetes pod termination, deployment)

func main() {
    srv := &http.Server{Addr: ":8080", Handler: handler}

    // Listen for shutdown signal
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)

    go srv.ListenAndServe()

    <-quit // block until signal
    log.Println("shutting down...")

    // Graceful shutdown: wait for active connections
    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()

    if err := srv.Shutdown(ctx); err != nil {
        log.Printf("forced shutdown: %v", err)
    }
    log.Println("server stopped cleanly")
}

// Kubernetes: preStop hook + terminationGracePeriodSeconds
// spec.containers[].lifecycle.preStop.exec.command = ["sleep", "10"]
// spec.terminationGracePeriodSeconds = 60

// Load balancer: stop sending new traffic before SIGTERM
// readinessProbe: return 503 on shutdown to drain LB faster
```

Q127. What is network throughput optimization?**Difficulty:**
Medium

Bandwidth vs Throughput vs Latency:

Bandwidth: maximum capacity (e.g., 1 Gbps link)

Throughput: actual data transferred per second (often less than bandwidth)

Latency: time for one packet to travel (RTT)

TCP throughput formula:

$\text{throughput} = \text{window_size} / \text{RTT}$

10MB window / 100ms RTT = 800 Mbps theoretical max

Optimizations:

1. Increase TCP window size

```
sysctl -w net.core.rmem_max=134217728 # 128MB
sysctl -w net.ipv4.tcp_rmem="4096 87380 134217728"
```
2. TCP congestion control

```
sysctl -w net.ipv4.tcp_congestion_control=bbr # better for high-latency
```
3. Jumbo frames (MTU 9000)
 Reduces CPU overhead for same bandwidth
 Requires network support (datacenter-only)
4. Connection pooling
 Fewer TCP handshakes = more bandwidth for data
5. Compression
 gzip/br for HTTP (CPU vs bandwidth trade-off)
6. HTTP/2 multiplexing
 Multiple streams on one connection
7. UDP + QUIC for latency-sensitive
 Avoid TCP HOL blocking

Q128. What is health check endpoint design?**Difficulty:**
Medium

// /healthz: is the process alive? (liveness)

// /readyz: is it ready to receive traffic? (readiness)

```
type HealthChecker struct {
    db      *pgxpool.Pool
    redis   *redis.Client
}
```

```
func (h *HealthChecker) Liveness(w http.ResponseWriter, r *http.Request) {
    // Just: is the process running and not deadlocked?
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}
```

```
func (h *HealthChecker) Readiness(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)
    defer cancel()
```

```

checks := map[string]string{}
allOk := true

if err := h.db.Ping(ctx); err != nil {
    checks["database"] = "unhealthy: " + err.Error()
    allOk = false
} else {
    checks["database"] = "ok"
}

if err := h.redis.Ping(ctx).Err(); err != nil {
    checks["redis"] = "unhealthy: " + err.Error()
    allOk = false
} else {
    checks["redis"] = "ok"
}

status := http.StatusOK
if !allOk { status = http.StatusServiceUnavailable }

w.WriteHeader(status)
json.NewEncoder(w).Encode(map[string]interface{}{"checks": checks})
}
// Kubernetes: livenessProbe → /healthz, readinessProbe → /readyz

```

Q129. What is TCP Nagle's algorithm and TCP_NODELAY?

Difficulty:
Medium

```

// Nagle's algorithm: buffer small packets into larger ones
// Goal: reduce "small packet problem" (many tiny TCP segments)
// Default: enabled in most OS

// Nagle: don't send data until:
// a) buffer is full (MSS: 1460 bytes), OR
// b) all unacknowledged data is ACKed

// Problem: adds latency for interactive protocols (SSH, gaming, gRPC)
// Example: 40ms+ added latency per small write in high-latency networks

// TCP_NODELAY: disable Nagle's algorithm
conn, _ := net.Dial("tcp", "host:port")
tcpConn := conn.(*net.TCPConn)
tcpConn.SetNoDelay(true) // disable Nagle

// HTTP servers in Go: TCP_NODELAY enabled automatically
// gRPC: enables TCP_NODELAY by default

// When to keep Nagle enabled:
// Bulk data transfer (file uploads): Nagle helps coalesce
// When bandwidth matters more than latency

// Related: TCP_CORK (Linux): explicitly control when to flush
// CORK=1 → buffer everything; CORK=0 → flush buffer

```

Q130. What is OSI model quick reference?

Difficulty:
Medium

Layer 7: Application – HTTP, HTTPS, DNS, SMTP, FTP, WebSocket
Layer 6: Presentation – TLS/SSL, encoding (JSON/XML), compression
Layer 5: Session – session management, RPC (rarely discussed)
Layer 4: Transport – TCP, UDP, port numbers, QUIC
Layer 3: Network – IP, ICMP, routing (BGP, OSPF), ARP
Layer 2: Data Link – Ethernet, WiFi (802.11), MAC addresses, switches
Layer 1: Physical – cables, optical fiber, radio waves, hubs

Protocols at each layer:

TCP/IP model (4 layers): Application, Transport, Internet, Network Access

Troubleshooting by layer:

L7: curl, HTTP response codes, application logs
L4: netstat, ss, telnet host port, tcpdump port N
L3: ping, traceroute, ip route
L2: arp, MAC tables on switches
L1: cable check, link light on NIC

Interview tip:

Load balancer: L4 (TCP) or L7 (HTTP) LB
AWS ALB: L7 (can route by path, header, method)
AWS NLB: L4 (faster, preserves client IP)
Firewall: L3/L4 (iptables, security groups)

Q131. What is gRPC-web and browser compatibility?

Difficulty:
Medium

Problem: gRPC uses HTTP/2 trailers → browsers can't use gRPC directly

Browser Fetch API: no access to HTTP trailers (required for gRPC status)

gRPC: status code in trailing headers → browser can't read

Solutions:

1. gRPC-Web:

Encodes gRPC in HTTP/1.1 compatible format

Proxy (Envoy, grpc-web-proxy) converts gRPC-Web → gRPC

Client: @improbable-eng/grpc-web or @protobuf-ts/grpc-web

Limitation: no client streaming or bidirectional streaming

2. gRPC-Gateway:

Generate REST+JSON from proto definitions

Client → REST/JSON → gRPC-Gateway → gRPC backend

Full browser compatibility

Separate HTTP/1.1 endpoint

3. Connect Protocol (Buf):

New protocol compatible with gRPC, gRPC-Web, and REST

Works natively in browsers without proxy

connect-go: Go server library

connect-web: TypeScript client

Recommended for new projects

4. HTTP/3 + QUIC:

Future: browsers support QUIC → gRPC over QUIC possible

Q132. What is network namespaces used by Docker and Kubernetes?**Difficulty:**
Medium

```
# Each Docker container: own network namespace
# Container sees: lo + eth0 (veth pair to docker0 bridge)
# Host sees: docker0 bridge + veth interfaces

# Inspect container network namespace
container_pid=$(docker inspect --format '{{.State.Pid}}' mycontainer)
nsenter -t $container_pid -n ip addr # run ip command in container's network ns

# Kubernetes pod: shared network namespace
# ALL containers in pod share: IP, port space, lo interface
# Different containers: share same IP, communicate via localhost!
kubectl exec -it pod-name -c container1 -- curl localhost:8080 # reaches container2

# Create custom network namespace (manual)
ip netns add myns
ip netns exec myns ip link list

# veth pair (virtual ethernet pair):
# One end in host namespace, other in container namespace
ip link add veth0 type veth peer name veth1
ip link set veth1 netns myns
# veth0 ↔ veth1: bidirectional pipe

# CNI (Container Network Interface): Kubernetes networking plugins
# Calico: BGP-based, network policy
# Flannel: VXLAN overlay, simple
# Cilium: eBPF-based, advanced observability + policy
```

Q133. What is HTTP streaming (chunked transfer encoding)?**Difficulty:**
Medium

```
// Chunked Transfer Encoding: send response body in chunks
// Use: streaming large responses, server-sent events, live logs

http.HandleFunc("/stream", func(w http.ResponseWriter, r *http.Request) {
    flusher, ok := w.(http.Flusher)
    if !ok {
        http.Error(w, "streaming not supported", 500)
        return
    }

    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")
    w.Header().Set("X-Accel-Buffering", "no") // disable nginx buffering

    for i := 0; i < 10; i++ {
        fmt.Fprintf(w, "data: event %d\n\n", i)
        flusher.Flush() // send chunk immediately
        time.Sleep(time.Second)
    }
})

// SSE (Server-Sent Events): built on chunked encoding
```

```
// Format: "data: <message>\n\n"
// Client (browser): new EventSource("/stream")
// One-directional: server → client
// Auto-reconnects on disconnect

// vs WebSocket:
// SSE: simpler, HTTP-based, one-direction, auto-reconnect
// WebSocket: bidirectional, custom protocol, must handle reconnect
// Use SSE for: live feed, notifications, progress updates
```

Q134. What is TCP SYN cookies for DDoS protection?

Difficulty:
Medium

SYN flood attack: send millions of SYN packets with spoofed IPs
Server allocates half-open connection state for each SYN
Connection table fills up → legitimate connections rejected

SYN cookies: stateless SYN-ACK without allocating connection state
Server: encodes connection info in SYN-ACK sequence number
seq_num = hash(client_ip, client_port, server_ip, server_port, time)
Client: sends ACK with seq_num+1
Server: validates ACK → only then allocate connection state

Linux SYN cookies:
net.ipv4.tcp_syncookies = 1 (default: enabled)
Automatically used when SYN backlog fills up

Other DDoS mitigations:
Rate limit SYN: iptables -A INPUT -p tcp --syn -m limit --limit 1/s -j ACCEPT
Cloud: Cloudflare, AWS Shield absorb at edge (anycast)
Blackhole routing: null-route attacking IP ranges via BGP
CAPTCHA: for HTTP-layer floods

Go application level:
Rate limit by IP (token bucket in Redis)
Connection timeout: reject if TLS handshake > 5s
Cloudflare proxy: absorb DDoS before reaching origin

Q135. What is packet loss and its impact on TCP?

Difficulty:
Medium

Packet loss effects on TCP:

TCP detects loss via:

1. Timeout: retransmit timer expires (slow, 1-3s)
2. Triple duplicate ACK: fast retransmit (faster, ~1 RTT)

Congestion control reaction:
Loss detected → assume congestion → halve cwnd (slow start)
Recovery: slow start or fast recovery

Impact:

- 1% packet loss: TCP throughput drops 10-50%
- 5% packet loss: connection barely usable

Bandwidth-Delay Product (BDP):
BDP = bandwidth × RTT

$100 \text{ Mbps} \times 100\text{ms} = 10\text{MB}$
TCP window must be $\geq$ BDP to fully utilize link

Measuring:

ping -c 100 host → count lost packets in summary
iperf3 --logfile results.txt → throughput with loss
mtr: continuous measurement per hop

UDP under packet loss:

No congestion control → maintain send rate regardless of loss
Use: video streaming (lost frame → skip, not retransmit)
QUIC: has its own congestion control

BBR (Bottleneck Bandwidth and RTT):

Google's congestion control algorithm
Doesn't use packet loss as signal (unlike CUBIC)
Estimates bandwidth and RTT directly
Better performance on lossy links (e.g., WiFi, mobile)
Enable: `sysctl -w net.ipv4.tcp_congestion_control=bbr`

Q136. What is Envoy proxy and service mesh?

Difficulty:
Medium

Envoy: high-performance, L7 proxy (written in C++)
Used as: sidecar in Istio, standalone edge proxy, API gateway

Features:

HTTP/1.1, HTTP/2, HTTP/3, gRPC
TLS termination + mTLS (mutual TLS)
Load balancing: round-robin, least-request, random, ring-hash
Circuit breaking: track errors, open circuit on threshold
Outlier detection: eject unhealthy hosts from load balancer
Retry: automatic with configurable conditions
Timeout: per-route timeouts
Rate limiting: local or global (external rate limit service)
Observability: Prometheus metrics, distributed tracing, access logs

Service mesh (Istio):

Control plane (istiod): configures all Envoy sidecars
Data plane: Envoy sidecars handle all traffic
Benefits:

mTLS between all services (zero-trust networking)
Distributed tracing (automatic span propagation)
Traffic management (canary, A/B, circuit breaking)
Observability (golden signals for every service)

Alternatives:

Linkerd: lightweight, Go-based, simpler than Istio
Consul Connect: HashiCorp, integrates with Consul service discovery
Cilium: eBPF-based, no sidecar needed (kernel-level)

Q137. What is TLS certificate management automation?

Difficulty:
Medium

Manual cert management: error-prone, expensive, security risk
Automation: cert-manager, Let's Encrypt, Vault PKI
cert-manager (Kubernetes):

```
# Automatically issues and renews TLS certificates
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.yaml

# Issuer: Let's Encrypt
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: {name: letsencrypt-prod}
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef: {name: letsencrypt-prod}
    solvers: [{http01: {ingress: {class: nginx}}}]

# Certificate: auto-issued and renewed
apiVersion: cert-manager.io/v1
kind: Certificate
metadata: {name: myapp-tls}
spec:
  secretName: myapp-tls-secret
  issuerRef: {name: letsencrypt-prod, kind: ClusterIssuer}
  dnsNames: [myapp.example.com]
# cert-manager renews 30 days before expiry automatically

# ACME challenges:
# HTTP-01: server serves token at /.well-known/acme-challenge/
# DNS-01: create TXT record (supports wildcard certs)
```

Q138. What is IPv6 dual-stack deployment?

Difficulty:
Medium

```
// Dual-stack: server listens on both IPv4 and IPv6
// Go: automatically dual-stack when binding to :: (IPv6 wildcard)

// Listen on both IPv4 and IPv6
server := &http.Server{
  Addr: ":8080", // equivalent to 0.0.0.0:8080 + [::]:8080 on dual-stack systems
}
// Note: on Linux, [::]:8080 accepts both IPv4 and IPv6 (via IPv4-mapped addresses)
// e.g., client 192.168.1.5 appears as ::ffff:192.168.1.5

// Explicit dual-stack (separate listeners):
ln4, _ := net.Listen("tcp4", ":8080")
ln6, _ := net.Listen("tcp6", ":8080")

// Dial: automatically prefers IPv6 (Happy Eyeballs, RFC 6555)
conn, _ := net.Dial("tcp", "example.com:80")
// Go resolver tries IPv6 first (AAAA record), falls back to IPv4 (A record)

// Kubernetes: dual-stack clusters (k8s 1.21+)
// Pod gets both IPv4 and IPv6 addresses
// Service: spec.ipFamilies: [IPv4, IPv6]

// Check: if server is listening on IPv6
ss -tlnp | grep ">:::8080"
// or: ss -tlnp | grep "0.0.0.0:8080"
```

Q139. What is network policy for zero-trust?**Difficulty:**
Medium

```
# Zero-trust: deny all by default, whitelist explicitly
# Kubernetes NetworkPolicy: L3/L4 filtering between pods
```

```
# Default deny all ingress + egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {} # selects ALL pods
  policyTypes: [Ingress, Egress]
```

```
# Allow only specific traffic (whitelist)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-to-db
spec:
  podSelector:
    matchLabels: {app: postgres}
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector:
          matchLabels: {app: api}
      ports:
        - port: 5432
```

```
# Allow DNS egress (required for service discovery)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: allow-dns}
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - ports:
      - port: 53
        protocol: UDP
      - port: 53
        protocol: TCP
```

Q140. What is load balancer health checks?**Difficulty:**
Medium

Health check types:

TCP health check:

- Just tries to connect (TCP handshake)
- Pass: connection succeeds
- Fail: connection refused or timeout
- Fast, low overhead, no app-level check

HTTP health check:

GET /healthz → check response code (200 = healthy)
Can check: DB connectivity, cache, dependencies
Better than TCP: verifies app is actually serving requests
Overhead: real HTTP request every N seconds

gRPC health check:

Standard: grpc.health.v1.Health/Check
google.golang.org/grpc/health
Import: grpc_health_v1.RegisterHealthServer(s, h)
status: SERVING, NOT_SERVING, SERVICE_UNKNOWN

Parameters:

Interval: 30s (check every 30 seconds)
Timeout: 5s (fail if no response in 5s)
Unhealthy threshold: 2 (2 consecutive failures = unhealthy)
Healthy threshold: 3 (3 consecutive passes = healthy again)

AWS ALB:

healthCheckPath: /healthz
healthCheckIntervalSeconds: 30
healthyThresholdCount: 2
unhealthyThresholdCount: 3

Graceful shutdown:

/readyz returns 503 → LB stops sending traffic → SIGTERM → drain

Q141. What is multiplexing vs connection per request?

Difficulty:
Medium

HTTP/1.0: one TCP connection per request
Overhead: 3-way handshake + TLS handshake per request
Latency: 1.5-2 RTT overhead per request

HTTP/1.1: connection reuse (keep-alive)
One TCP connection for multiple sequential requests
Problem: HOL blocking (request 2 waits for response 1)
Pipelining: send N requests without waiting, but responses must be in order

HTTP/2: multiplexing
Multiple concurrent streams on ONE connection
Stream: independent logical request/response
No HOL blocking at HTTP layer (still TCP HOL blocking)
Header compression: HPACK

HTTP/3: no TCP HOL blocking
QUIC: stream-level retransmit (only that stream blocked on loss)
Better on lossy networks

gRPC multiplexing:
Many concurrent RPCs on one HTTP/2 connection
Efficient for microservice communication
vs REST HTTP/1.1: gRPC 5-10x more concurrent requests per connection

Database connection multiplexing:
pgBouncer: many app connections → few DB connections
Transaction mode: connection released after each transaction

Q142. ### Q141-Q150: Final Networking Questions**Difficulty:**
Medium**Q143. What is QUIC handshake timing?****Difficulty:**
Medium

QUIC 1-RTT handshake (new connection):

Client → Initial (ClientHello + crypto data)
Server → Initial (ServerHello + cert + Handshake data)
Client → Handshake (Finished)
→ 1 RTT before first application data

QUIC 0-RTT (known server, session resumption):

Client → Initial + 0-RTT data (application data immediately!)
Server → receives 0-RTT data before finishing handshake
→ 0 RTT for application data (but replay attack risk)

vs TLS 1.3 over TCP:

TCP: 1 RTT (3-way handshake)
TLS 1.3: 1 RTT
Total: 2 RTT for first byte
QUIC: 1 RTT total (combines transport + TLS)

Performance advantage:

QUIC saves 1 RTT on every new connection
50ms RTT: saves 50ms per connection setup
Mobile: connection migration (no reconnect on IP change)

Q144. What is network observability with eBPF?**Difficulty:**
Medium

eBPF: run programs in Linux kernel (safely, verified)

No kernel patch needed
Use: networking, security, observability

Tools built on eBPF:

Cilium: networking (replaces kube-proxy) + network policy + observability
Pixie: auto-instrumentation (no code changes, captures all traffic)
Falco: security runtime monitoring (detect network anomalies)
bpftrace: ad-hoc eBPF programs for debugging

Capabilities:

Trace: every syscall, network packet, function call
Block: drop packets based on policy (L7-aware firewall)
Measure: latency, throughput per pod/service without sidecars
Map: network topology, service dependencies

Example: trace HTTP requests with bpftrace

```
bpftrace -e 'tracepoint:syscalls:sys_enter_sendto { printf("HTTP send: %d bytes\n", args->len); }'
```

vs sidecar proxy (Envoy):

eBPF: no extra process, kernel-level (lower overhead)
Envoy: user-space, mature, more features
Cilium: combines both (eBPF for basic networking, Envoy for L7)

Q145. What are common API gateway patterns?**Difficulty:**
Medium

```
// API Gateway: single entry point for all microservices
// Functions: auth, rate limiting, routing, transformation, caching

// Rate limiting middleware
func RateLimitMiddleware(next http.Handler) http.Handler {
    limiter := rate.NewLimiter(rate.Limit(100), 200) // 100/s, burst 200
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if !limiter.Allow() {
            w.Header().Set("Retry-After", "1")
            http.Error(w, "rate limit exceeded", http.StatusTooManyRequests)
            return
        }
        next.ServeHTTP(w, r)
    })
}

// Authentication middleware
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        claims, err := validateJWT(token)
        if err != nil { http.Error(w, "unauthorized", 401); return }
        ctx := context.WithValue(r.Context(), ctxUserKey, claims)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// Popular API gateways:
// Kong: Lua-based, plugin ecosystem, self-hosted
// AWS API Gateway: managed, expensive at scale
// nginx + lua: high performance, custom logic
// Traefik: cloud-native, Kubernetes-aware, auto-config
// Envoy: L7 proxy, used in service meshes
```

Q146. What is TLS session resumption?**Difficulty:**
Medium

TLS session resumption: reuse crypto material from previous connection
 Avoids full TLS handshake on reconnect (saves 1 RTT)

Session ID (TLS 1.2):

- Server assigns session ID on first connection
- Client sends session ID on reconnect → server resumes
- Server must store session state (stateful, memory pressure)

Session Tickets (TLS 1.2):

- Server sends encrypted ticket (client stores it)
- Client sends ticket on reconnect → server decrypts, resumes
- Stateless: server doesn't store session state
- Key rotation: rotate ticket encryption key regularly

TLS 1.3:

- Uses PSK (Pre-Shared Key) for resumption
- Server sends NewSessionTicket after handshake

Client uses PSK on reconnect → 1-RTT or 0-RTT

0-RTT (TLS 1.3 / QUIC):

- Client sends application data immediately with PSK
- Server processes before verifying (replay attack risk!)
- Safe for: idempotent requests (GET, cacheable)
- Dangerous for: mutations (POST/DELETE)
- Mitigation: anti-replay token, limit 0-RTT to safe operations

Go TLS config:

```
tls.Config{SessionTicketsDisabled: false} // tickets enabled by default
```

Q147. What is multicast and anycast networking?

Difficulty:
Medium

Unicast: one-to-one (normal traffic)

Broadcast: one-to-all (same network, rare in internet)

Multicast: one-to-many (subscribed receivers)

Anycast: one-to-nearest (multiple servers, same IP)

Multicast (IP multicast):

- Range: 224.0.0.0/4 (IPv4 multicast)
- Protocol: IGMP (join/leave multicast groups)
- Use: streaming video to many receivers, IoT sensors
- Requirement: routers must support multicast routing (PIM)
- Rarely used on internet (ISPs don't support it)

Anycast:

- Same IP announced via BGP from multiple locations
- Network routes to "nearest" (lowest BGP metric)
- Use: DNS root servers, Cloudflare CDN (1.1.1.1), DDoS protection
- Examples:
 - 8.8.8.8 (Google DNS): anycast to nearest Google datacenter
 - 1.1.1.1 (Cloudflare): anycast to nearest Cloudflare PoP
 - Cloudflare: all customer IPs are anycast

Anycast failover:

- Node fails → withdraw BGP announcement
- Traffic reroutes to next nearest node
- Convergence: 30-60 seconds (BGP convergence time)
- vs DNS failover: 5-30 minutes (DNS TTL)

Anycast for DDoS:

- Attack traffic spread across all PoPs (not concentrated)
- Each PoP absorbs fraction of attack

Q148. What is traffic shaping and QoS?

Difficulty:
Medium

QoS (Quality of Service): prioritize critical traffic

Use: ensure video calls/VoIP aren't disrupted by file downloads

Traffic shaping (egress):

- Token bucket: burst to max rate, sustained at average
- Leaky bucket: smooth output (constant rate)
- HTB (Hierarchical Token Bucket): Linux qdisc

Linux traffic control (tc):

```
tc qdisc add dev eth0 root tbf rate 100mbit burst 32kbit latency 400ms
```

Congestion avoidance:

FIFO: first in first out (no QoS)

WFQ: weighted fair queuing (weights per flow)

RED (Random Early Detection): drop packets probabilistically before queue fills

ECN (Explicit Congestion Notification): mark instead of drop (TCP reacts)

Priority queuing in Kubernetes:

LimitRange: CPU/memory requests and limits per namespace

PriorityClass: higher priority pods evict lower priority

ResourceQuota: namespace-level resource limits

HPA: autoscale based on CPU/memory/custom metrics

Application-level:

Rate limiting in API gateway (per client)

Priority queue for background jobs (don't starve foreground)

Concurrency limits: semaphore per service

Q149. What is SSL pinning?

Difficulty:
Medium

SSL pinning: hard-code expected certificate in client

Prevents MITM even if attacker has trusted CA cert

Use: mobile apps, IoT devices, high-security APIs

Types:

Certificate pinning: pin exact cert (must update when cert rotates)

Public key pinning (HPKP): pin public key (survives cert renewal if same key)

SPKI pinning: pin SubjectPublicKeyInfo hash

In Go HTTP client:

```
config := &tls.Config{
    VerifyPeerCertificate: func(rawCerts [][]byte, verifiedChains [][]*x509.Certificate) error {
        cert, _ := x509.ParseCertificate(rawCerts[0])
        pubKeyHash := sha256.Sum256(cert.RawSubjectPublicKeyInfo)
        expected := "sha256//base64encodedHash=="
        if base64.StdEncoding.EncodeToString(pubKeyHash[:]) != expectedHash {
            return errors.New("certificate pin mismatch")
        }
        return nil
    },
}
```

Risks:

App broken if cert rotated without updating pin

Solution: pin backup key (always have 2 pins)

Monitoring: HPKP had Report-URI to detect failures

Deprecation:

HTTP HPKP header deprecated (Chrome removed, too risky)

Certificate Transparency (CT logs) is better alternative for browsers

Pinning still valid for native apps (mobile)

Q150. What is Istio traffic management?

Difficulty:
Medium

```
# VirtualService: define routing rules
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: {name: orders-vs}
spec:
  hosts: [orders]
  http:
    - match:
      - headers:
          x-canary: {exact: "true"}
      route:
        - destination: {host: orders, subset: v2}
    - route:
        - destination: {host: orders, subset: v1}
          weight: 90
        - destination: {host: orders, subset: v2}
          weight: 10 # 10% canary traffic

# DestinationRule: load balancing + circuit breaking per subset
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata: {name: orders-dr}
spec:
  host: orders
  trafficPolicy:
    connectionPool:
      tcp: {maxConnections: 100}
      http: {http2MaxRequests: 1000, pendingRequests: 100}
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
  subsets:
    - name: v1
      labels: {version: v1}
    - name: v2
      labels: {version: v2}
```

Q151. What is network debugging in production?

Difficulty:
Medium

```
# Without disrupting traffic:

# 1. Check connectivity
curl -v --max-time 5 https://service:8080/healthz
curl -v --connect-timeout 3 --max-time 10 http://backend/api

# 2. DNS resolution
dig +short service.namespace.svc.cluster.local
nslookup service 10.96.0.10 # query specific DNS server

# 3. TCP connectivity
nc -zv hostname 443 # check if port open
timeout 5 bash -c ">/dev/tcp/hostname/443" && echo open || echo closed

# 4. Capture traffic (read-only)
tcpdump -i eth0 -n port 8080 -c 100 # capture 100 packets on port 8080
```

```
tcpdump -i eth0 -n host 10.0.0.5      # traffic to/from specific IP

# 5. Check TLS
openssl s_client -connect hostname:443 -servername hostname
# Shows: certificate chain, protocol version, cipher suite

# 6. Kubernetes specific
kubectl exec pod -- curl -v http://other-service/ # cross-pod connectivity
kubectl logs pod --previous                       # logs from crashed container
kubectl describe pod pod-name                     # events, conditions

# 7. Network policy check
# If curl fails with "connection refused" → app not listening
# If curl fails with "connection timeout" → network policy blocking
# If curl fails with "no route to host" → DNS or routing issue
```

Q152. What is networking production readiness checklist?

Difficulty:
Medium

Security:

- TLS 1.2+ only (disable TLS 1.0, 1.1)
- Strong cipher suites (ECDHE-based, no RC4/DES)
- HSTS: Strict-Transport-Security header
- Certificate auto-renewal (cert-manager or ACM)
- mTLS for service-to-service communication
- NetworkPolicy: default deny, whitelist explicitly
- API rate limiting at gateway

Performance:

- HTTP/2 enabled (or HTTP/3 via Cloudflare)
- Keep-alive connections (connection pooling)
- CDN for static assets (cache-hit ratio > 90%)
- Compression: gzip/brotli for HTTP responses
- TCP_NODELAY for latency-sensitive connections
- DNS caching (ndots:5 → consider reducing for K8s)

Reliability:

- Health checks configured (liveness + readiness)
- Graceful shutdown (drain in-flight requests)
- Circuit breakers for external dependencies
- Retry with exponential backoff
- Timeouts on all outbound connections
- Connection pool sizing matches expected concurrency

Observability:

- Access logs with latency, status, bytes
- Distributed tracing (OpenTelemetry)
- Metrics: request rate, error rate, latency (p50/p99/p999)
- Alerts: latency > SL0, error rate > threshold
- Network topology map (who talks to whom)